

Network / PIJ 方法归属重构方案

根据反馈收紧后的第二版实施计划

当前代码核查、文件归属判断与 Codex 实施边界

基线: `code(2).zip`, 重点目录为 `mignet_ce/networks` 与 `mignet_ce/pij`

本版以用户最新反馈为最终约束: **不改名、不移动 `common.py` 与 `cost_fusion.py`; 不整理 `legacy`; 不建立额外的方法共享架构。** 仅保留三项实际重构: 把 Network 私有 GRN 逻辑并回所属方法、把 block KL 交还 `compare_N_kl`、把纯数学工具按原文件移动到一个最小化的内部目录。

生成日期: 2026 年 7 月 20 日

项目: 2026 因果涌现

文档类型: Codex 实施计划 / 代码结构验收依据 (反馈修订版)

重构原则: 不改变算法、不改变 registry key、不改变默认参数与输出语义

目录

1	结论先行：本版只做三类修改	2
2	反馈决策矩阵	2
3	Network：把 GRN 私有逻辑并回所属方法	3
3.1	当前问题	3
3.2	目标修改	3
4	PIJ：把 block KL 交还 compare_N_kl	4
4.1	为什么 block KL 是方法私有逻辑	4
4.2	最小化改造方式	4
4.3	必须保持的数值语义	5
5	Compare 数学工具：只移动，不重写	6
5.1	本步骤的准确边界	6
5.2	为什么本版不再合并 distances/cosine/kl	6
5.3	允许修改的内容	7
6	明确保持原样的部分	7
6.1	common.py	7
6.2	cost_fusion.py	7
6.3	legacy	7
6.4	registry 与 package export	8
7	修订后的目标目录	8
7.1	Network	8
7.2	PIJ compare	8
7.3	PIJ legacy	9
8	文件级 Codex 实施步骤	9
8.1	步骤 0：建立重构前快照	9
8.2	步骤 1：合并 Network GRN 私有逻辑	9

8.3	步骤 2: 把 block KL 交还 compare_N_kl	10
8.4	步骤 3: 移动五个数学工具文件	10
8.5	步骤 4: 只清理本轮产生的旧路径	10
8.6	步骤 5: 验证 registry, 而不是重构 registry	11
9	测试计划	11
9.1	导入与编译测试	11
9.2	Network 数值回归	11
9.3	compare_N_kl 回归	11
9.4	数学工具移动回归	12
9.5	legacy 保护性测试	12
10	Codemod 不得做的事情	12
11	完成标准	13
12	最终建议	13

1. 结论先行：本版只做三类修改

根据最新反馈，本轮重构不再追求统一所有目录风格，也不再把所有辅助文件重新分层。目标从“全面整理”收紧为“修复真正的归属问题，同时尽量少动代码”。

最终保留的三项修改：

- 1. 将 `mignet_ce/networks/grn_state.py` 并入 `mignet_ce/networks/light_cci_grn.py`;
- 2. 将 `mignet_ce/pij/compare/block_kl.py` 的逻辑交还 `mignet_ce/pij/compare/compare_N_kl.py`;
- 3. 将 `compare` 中纯数学、纯特征和纯优化器工具按原文件移动到最小化的 `compare/_shared/`，不合并公式、不重写实现。

明确取消的修改：

- 不把 `common.py` 改名为 `base.py`;
- 不把 `cost_fusion.py` 改名为 `cost_fusion_base.py`;
- 不移动或重组 `pij/legacy` 下的任何文件;
- 不重做 `registry` 结构，不调整 `registry key`，不扩大 `package export`;
- 不再建立覆盖 `PIJ` 全体系的统一 `shared/base` 架构。

这里需要特别区分“文件不移动”和“文件绝对不编辑”。`common.py` 的路径、名称和总体职责保持不变，但为了把 `block KL` 从公共流程中交还给 `compare_N_kl`，允许对 `common.py` 做一处最小化的 `hook` 改造；除此之外，不对其结构做清理或拆分。

2. 反馈决策矩阵

对象	最终决定	本版执行方式
<code>networks/grn_state.py</code>	合并	并入 <code>light_cci_grn.py</code> ，随后删除原文件。
<code>compare/block_kl.py</code>	合并	并入 <code>compare_N_kl.py</code> ，由该方法覆盖通用 <code>KL cost hook</code> 。
<code>compare/common.py</code>	保留	文件名、路径和基类职责不变；仅为 <code>block KL</code> 归属调整加入最小 <code>hook</code> 。

对象	最终决定	本版执行方式
compare/cost_fusion.py	完全保留	不改名、不移动、不拆分，不借本任务清理其内部结构。
compare 数学工具	移动	features.py、distances.py、cosine.py、kl.py、sparse_ot.py 原样移入 compare/_shared/。
pij/legacy/*	不处理	保持现在的下划线内部模块与导入关系，不建立 legacy/_shared。
registry	不改结构	key、类映射和注册入口全部保持；理论上无需改 registry 文件。
package export	不扩大	只在确有旧路径 import 时修正，不新增公共暴露。

3. Network：把 GRN 私有逻辑并回所属方法

3.1 当前问题

mignet_ce/networks/grn_state.py 本身不是 registry 中可选择的 network method，但其内容不是通用 Network 工具，而是 light_cci_grn 构造 GRN 状态时使用的整套私有实现，包括：

- PreparedGRN 与 GRNStateResult；
- GRN 输入读取、基因与表达对齐；
- 双端表达门控；
- 确定性随机投影；
- projected GRN state 的构造与 metadata 写入。

它只被 light_cci_grn.py 直接使用；joint_cci_grn.py 对这套逻辑的使用来自继承链，而不是一个平行的通用调用者。因此其所有者仍然是 light_cci_grn.py。

3.2 目标修改

将 grn_state.py 的代码原样迁入 light_cci_grn.py。推荐的文件内部顺序为：

1. import；
2. GRN 相关 dataclass；

3. 输入准备和对齐函数；
4. 双端表达门控函数；
5. 投影和 state 构造函数；
6. LightCCIGRNNetworkBuilder 类。

迁移后删除：

```
mignet_ce/networks/grn_state.py
```

并移除 `light_cci_grn.py` 中对旧模块的 `import`。 `joint_cci_grn.py` 的继承关系、类名、registry key 与行为不变。

本步骤只改变代码位置。随机投影的 seed、哈希规则、矩阵维度、双端门控公式、稀疏矩阵类型、metadata key 和异常处理必须逐行保持。

4. PIJ：把 block KL 交还 compare_N_kl

4.1 为什么 block KL 是方法私有逻辑

当前 `block_kl.py` 虽然由 `common.py` 调用，但触发条件实际绑定到：

```
self.pij_key == "kl"
and self.feature_keys == ("N",)
and feature_set.metadata["grn_block"]["enabled"]
```

这意味着它只改变注册方法 `compare_N_kl` 在 GRN block 开启时的行为。用户打开 `compare_N_kl.py` 时却只能看到方法名、特征键和距离类型，看不到该方法最关键的特殊成本构造，这正是需要修复的归属问题。

4.2 最小化改造方式

`common.py` 不移动、不改名。只在现有 `ComparePijMethodBase` 中增加一个通用 KL cost hook，让普通方法继续使用当前实现：

```
class ComparePijMethodBase:
    def build_kl_cost(
        self,
        source,
        target,
        *,
        feature_set,
        side,
```

```

        pair,
        **kwargs,
    ):
        return pairwise_feature_kl(source, target, beta=...)

```

随后让 CompareNKLpijMethod 覆盖这一 hook，并把 block_kl.py 的实现放在同一方法文件中：

```

class CompareNKLpijMethod(ComparePijMethodBase):
    name = "compare_N_kl"
    feature_keys = ("N",)
    pij_key = "kl"

    def build_kl_cost(self, source, target, *, feature_set, **kwargs):
        block_meta = feature_set.metadata.get("grn_block", {})
        if not block_meta.get("enabled", False):
            return super().build_kl_cost(
                source, target, feature_set=feature_set, **kwargs
            )
        return self._build_block_kl_cost(
            source, target, feature_set=feature_set, **kwargs
        )

```

原 build_block_kl_cost() 可改为该类的私有静态方法，也可作为 compare_N_kl.py 内部以下划线开头的模块函数。关键不是形式，而是它必须和所属注册方法处于同一文件。

4.3 必须保持的数值语义

- GRN block 关闭时，结果与普通 N-only KL 路径完全一致；
- weight_g=0 时继续走当前 N-only exact fallback；
- 双 block 开启时，仍按当前规则分别做稳健归一化后加权；
- diagnostics 的 key、含义和数值保持；
- 不改变 KL beta、epsilon、裁剪方式或候选边。

完成后删除：

```
mignet_ce/pij/compare/block_kl.py
```


5.3 允许修改的内容

只允许：

- 修改 import 路径；
- 处理移动后必要的相对/绝对导入；
- 为避免循环 import 做最小顺序调整；
- 删除移动前的旧文件。

不允许重命名函数、合并公式、改变 dtype、修改归一化、优化向量化实现或调整异常条件。

6. 明确保持原样的部分

6.1 common.py

mignet_ce/pij/compare/common.py 保持当前名称和路径。它继续承担普通 compare 方法的统一执行框架。除 block KL hook 所需的最小修改外，不做：

- 改名；
- 拆文件；
- artifact 导出函数迁移；
- 统一方法配置重构；
- 大规模清理分支。

6.2 cost_fusion.py

mignet_ce/pij/compare/cost_fusion.py 完全保留。十个 costmix/euclidean 方法继续继承当前类，不改文件名、不改 import 语义，只需要在其内部引用数学工具时更新为新的 _shared 路径。

6.3 legacy

mignet_ce/pij/legacy/_ot_common.py 与 mignet_ce/pij/legacy/_developmental_ot_components.py 已经通过下划线表达内部模块，并被多个 legacy 方法使用。本轮不移动、不改名、不建立 legacy/_shared，也不清理 legacy 导入。

6.4 registry 与 package export

networks/registry.py 和 pij/registry.py 的 key、映射类、导入入口都不应变化。由于本轮移动的 PIJ 数学文件本来就不是 registry 入口，正常情况下 registry 文件无需编辑。

__init__.py 只在存在对旧数学工具路径的显式导出时做必要修正；不借此把 _shared 中的内容重新公开到 package 顶层。

7. 修订后的目标目录

7.1 Network

```
mignet_ce/networks/
|-- __init__.py
|-- base.py
|-- registry.py
|-- light_cci.py
|-- light_cci_grn.py      # 内含原 grn_state.py 私有逻辑
|-- joint_cci_grn.py
`-- ... 其他已注册 network 方法 ...

# 删除 grn_state.py
```

7.2 PIJ compare

```
mignet_ce/pij/compare/
|-- __init__.py
|-- common.py            # 保持名称；仅增加通用 KL hook
|-- cost_fusion.py       # 保持名称与职责
|-- compare_N_kl.py      # 内含 block KL 私有实现
|-- compare_E_cos.py
|-- compare_L_sot.py
|-- ... 其他注册方法文件 ...
|-- compare_main_lap_sr_spatial_sot.py
`-- _shared/
    |-- __init__.py
    |-- features.py
    |-- distances.py
    |-- cosine.py
    |-- kl.py
    `-- sparse_ot.py

# 删除 block_kl.py
```

```
# 删除 compare 根目录下五个数学工具的旧副本
```

7.3 PIJ legacy

```
mignet_ce/pij/legacy/  
|-- _ot_common.py  
|-- _developmental_ot_components.py  
`-- ... 所有现有 legacy 方法文件 ...  
  
# 本轮完全不改变
```

8. 文件级 Codex 实施步骤

8.1 步骤 0：建立重构前快照

在改动前保存：

1. NETWORK_BUILDERS.keys() 排序结果；
2. PIJ_METHOD_REGISTRY.keys() 排序结果；
3. 每个 key 对应的类名与 name；
4. 关键默认配置；
5. 小型 fixture 上代表方法的 cost、P 矩阵和 metadata。

代表方法至少包括：light_cci_grn、joint_cci_grn、compare_N_kl、compare_L_sot、一个 costmix 方法和一个 legacy OT 方法。

8.2 步骤 1：合并 Network GRN 私有逻辑

1. 将 grn_state.py 的 dataclass 与函数原样复制到 light_cci_grn.py；
2. 删除旧 import；
3. 删除 grn_state.py；
4. 全仓库搜索旧路径；
5. 对 light_cci_grn 和 joint_cci_grn 做数值回归。

8.3 步骤 2：把 block KL 交还 compare_N_kl

1. 在 `common.py` 增加默认 `build_kl_cost()` hook;
2. 让普通 KL 路径通过该 hook 调用原有 KL 实现;
3. 在 `compare_N_kl.py` 覆盖 hook;
4. 将 block KL 函数原样迁入该方法文件;
5. 删除 `common.py` 中针对 `feature_keys=="N",` 的隐藏特判;
6. 删除 `block_kl.py`;
7. 分别测试 block 开启、block 关闭与 `weight_g=0`。

8.4 步骤 3：移动五个数学工具文件

1. 创建 `compare/_shared/` 与空的 `__init__.py`;
2. 原样移动 `features.py`、`distances.py`、`cosine.py`、`kl.py`、`sparse_ot.py`;
3. 更新 `common.py`、`cost_fusion.py`、主方法和其他直接调用者的 `import`;
4. 不合并文件，不改函数签名;
5. 删除 `compare` 根目录的旧副本。

8.5 步骤 4：只清理本轮产生的旧路径

清理范围限定为已删除或已移动文件：

```
grep -R "networks.grn_state" -n mignet_ce scripts tests

grep -R "pij.compare.block_kl" -n mignet_ce scripts tests

grep -R "pij.compare.features" -n mignet_ce scripts tests
grep -R "pij.compare.distances" -n mignet_ce scripts tests
grep -R "pij.compare.cosine" -n mignet_ce scripts tests
grep -R "pij.compare.kl" -n mignet_ce scripts tests
grep -R "pij.compare.sparse_ot" -n mignet_ce scripts tests
```

不搜索并替换 `pij.compare.common`、`pij.compare.cost_fusion` 或 `legacy` 路径，因为它们本轮保持不变。

8.6 步骤 5：验证 registry，而不是重构 registry

本步骤只做检查，不做结构设计：

```
assert sorted(before_network_keys) == sorted(after_network_keys)
assert sorted(before_pij_keys) == sorted(after_pij_keys)
assert before_key_to_class_name == after_key_to_class_name
```

若 registry 文件因间接 import 无法加载，优先修复被移动模块的 import 路径，不能通过改 key 或删除方法绕过。

9. 测试计划

9.1 导入与编译测试

```
python -m compileall -q mignet_ce scripts
python -c "import mignet_ce"
python -c "from mignet_ce.networks.registry import NETWORK_BUILDERS"
python -c "from mignet_ce.pij.registry import PIJ_METHOD_REGISTRY"
```

还需遍历所有 registry key 完成实例化，以捕获移动数学模块后产生的漏改 import 和循环 import。

9.2 Network 数值回归

固定输入和 seed，比较重构前后：

- GRN 对齐后的基因顺序；
- 双端门控矩阵；
- deterministic projection matrix；
- grn_state_csr 的 shape、indices、indptr 与 data；
- light_cci_grn 和 joint_cci_grn metadata。

纯位置迁移原则上优先要求 bitwise equality。

9.3 compare_N_kl 回归

至少覆盖三种场景：

1. 没有 GRN block：应等于普通 N-only KL；
2. GRN block 开启且两块都有权重：总 cost、分块统计与 P 矩阵一致；
3. weight_g=0：exact fallback 与原结果一致。

9.4 数学工具移动回归

至少验证：

- 一个 cosine 方法；
- 一个 KL 方法；
- 一个 SOT 方法；
- 一个 costmix cosine 方法；
- 一个 costmix Euclidean 方法；
- `compare_main_lap_sr_spatial_sot`。

移动文件不应造成任何公式、dtype、稀疏结构或 diagnostics 差异。

9.5 legacy 保护性测试

虽然 legacy 不做结构修改，仍应选一个 legacy OT smoke test，确认 compare 模块移动没有通过公共 import 意外影响 legacy。该测试是保护性检查，不代表整理 legacy。

10. Codex 不得做的事情

1. 不得把 `common.py` 或 `cost_fusion.py` 改名、移动或拆分；
2. 不得创建 `legacy/_shared`，不得移动 legacy 内部模块；
3. 不得合并 `distances.py`、`cosine.py` 和 `kl.py`；
4. 不得把数学工具复制进每个方法文件；
5. 不得改变任何 network/PIJ registry key 或删除薄方法入口；
6. 不得借 block KL 归属调整修改 KL 公式、权重或归一化；
7. 不得借 GRN 文件合并修改门控、投影、维度或 metadata；
8. 不得扩大 `__init__.py` 的公共导出面；
9. 不得顺手清理 legacy、velocity OT、历史方法或实验矩阵；
10. 不得在本任务中做性能优化和大规模命名统一。

11. 完成标准

本次重构同时满足以下条件才算完成：

1. `grn_state.py` 已删除，其全部私有逻辑能在 `light_cci_grn.py` 直接阅读；
2. `block_kl.py` 已删除，`compare_N_kl.py` 明确拥有 block KL 行为；
3. `common.py` 与 `cost_fusion.py` 的文件名和路径保持；
4. 五个数学工具已原样移动到 `compare/_shared/`，没有文件合并；
5. `legacy` 目录结构和导入路径没有变化；
6. `registry key`、类映射、默认配置和可实例化性完全一致；
7. 关键方法的 `cost`、`P` 矩阵、`metadata` 与导出语义回归通过；
8. 全仓库没有本轮被删除/移动路径的残留 `import`；
9. `compileall`、`registry import` 与小型 `smoke test` 全部通过。

12. 最终建议

这一版不再追求“目录看起来绝对统一”，而是只处理真正妨碍方法可追踪性的对象：

- 方法私有业务逻辑回到方法文件：GRN state 回到 `light_cci_grn`, block KL 回到 `compare_N_kl`；
- 已有执行框架保持：`common.py` 和 `cost_fusion.py` 不动；
- 纯工具只做轻量归档：五个数学文件移到最小化的 `_shared`，但不重写、不合并；
- 旧体系不扩大改动：`legacy` 和 `registry` 只验证，不重构。

一句话概括：能明确归属于单一方法的逻辑就并回方法；已经稳定工作的公共执行框架保持原样；真正无业务归属的数学工具只移动目录，不改变实现。

13. 新增研究推进方案：从当前最好结果到可解释证据

本节及其后续章节是在前述代码重构方案已经锁定的前提下追加的研究计划。**原报告第 1–12 节的内容、顺序、模板和实施边界全部保持不变。**本次只保留当前最好结果的归因实验，用于判断现有结果是否真正依赖真实 GRN feature。

当前事实起点：

- 当前唯一作为最好结果保留的是 `light_cci_grn+compare_N_kl`；现有结果文件包含 18 项实验，其中 14 项 `EI_gain` 为正。
- 原始 `light_cci` 的结果不理想，但本次不擅自引用其他历史结果，也不在报告中补造不存在的横向数值比较。
- 当前 `light_cci_grn` 继承 LightCCI 的图构建逻辑，主要把 GRN state 写入 graph meta-data；`compare_N_kl` 再在 PIJ 成本层融合 N block 与 G block。因此当前最好结果应首先被作为“feature 如何影响因果涌现”的候选证据来检验。

13.1 本轮只推进一个阶段

1. **阶段一：解释当前最好结果。**固定 LightCCI 图结构和现有 N/G block-KL 主方法，只做最小受控消融，判断结果改善是否真正来自 GRN feature。

本次追加明确不包含：E、L、Sr 或 spatial 等新 feature；大规模权重、维数和温度搜索；纯 GRN 网络的独立因果涌现验证；feature-related 下游相关性分析。下游分析留到阶段一完成后再单独讨论和追加。

14. 阶段一：当前最好结果的最小归因实验

14.1 阶段目标

阶段一不再寻找新的“更好方法”，而是回答一个更窄的问题：

在 LightCCI 图结构、候选边、NMF 设置、GRN state 构造、KL 公式、PIJ 温度和实验任务全部一致时，当前结果是否必须依赖**真实 GRN feature**？

当前完整方法记为：

$$C_{\text{full}} = 0.5 \tilde{C}_N + 0.5 \tilde{C}_G,$$

其中 C_N 来自 CCI 邻接的 N 特征， C_G 来自表达门控后的 GRN state，两个 block 在完整方法中分别做稳健归一化后再加权。

14.2 必须固定的实验条件

四组实验必须共用：

- 同一个 `light_cci_grn network` 输入和同一批 18 项任务；
- 相同的候选边、NMF rank、GRN top-k、GRN projection seed 与 state 维数；
- 相同的 KL beta、PIJ 温度、裁剪规则和导出设置；
- 相同的 N/G 成本计算函数与稳健归一化实现；
- 除指定的 ablation mode 外，不允许同时改动其他超参数。

现有 `compare_N_kl` 的默认完整路径和当前结果必须原样保留。控制组应通过新增显式实验模式实现，不能为了得到对照而改变当前默认输出。

14.3 四组受控实验

组别	N	真实 G	随机 G	作用
Full：当前方法	是	是	否	当前最好结果，作为冻结的 anchor；权重保持 0.5/0.5。
N-only control	是	否	否	检验不使用 GRN feature 时的结果。为公平比较，N block 使用与 Full 中完全相同的稳健归一化。
G-only control	否	是	否	判断 GRN state 是否具有独立信息，还是只能与 N 互补。G block 使用与 Full 中完全相同的稳健归一化。
N + shuffled-G	是	否	是	在 unit 之间置换 G，保留维数和数值分布，检验 Full 的改善是否来自真实的 unit-GRN 对应。至少运行 5 个固定 seed。

14.4 最小代码追加边界

在完成前述归属重构之后，只在 `compare_N_kl.py` 所属的方法逻辑内增加显式的实验控制，不再新建游离数学文件。建议配置字段为：

```
grn_ablation_mode:  
    full  
    n_only_control  
    g_only_control
```

```
shuffle_g_control
```

```
grn_shuffle_seed: <int>
```

实施要求：

- 默认值必须是 full，因此旧命令、旧结果和默认数值语义不变；
- `n_only_control` 与当前 `weight_g=0` 的 exact fallback 不是同一个对照。前者用于公平归因，必须对 N block 使用 Full 中的相同归一化；后者作为历史兼容路径继续保留；
- `shuffle_g_control` 只置换 unit 轴，不改变 G 的列、维数和边际数值分布；source 和 target 各自只在同一时间点、同一层级内部置换；
- diagnostics、run config 和输出目录必须记录 ablation mode 与 seed，防止覆盖当前最好结果；
- 不在本阶段扫描 N/G 权重、projection 维数或温度。

14.5 阶段一的判定规则

- **Full 优于 N-only，且优于 shuffled-G：**真实 GRN feature 对当前结果有可归因的增量贡献。
- **Full 优于 N-only，但与 shuffled-G 接近：**改善更可能来自额外成本块、归一化或平滑效应，暂时不能归因于真实 GRN 生物信息。
- **G-only 较差，而 Full 最好：**说明 GRN 不必独立替代 CCI；更合理的结论是 N 与 G 具有互补性。
- **Full 在统一控制后不再稳定：**停止继续优化当前 feature 融合，先检查当前结果对归一化和 PIJ 平滑程度的依赖。

阶段一完成标准：不是把 14/18 调成更高，而是能够用 N-only、G-only 和 shuffled-G 三类对照解释当前 14/18 为什么出现。

15. 阶段一执行顺序与本轮停止边界

1. **冻结现有证据：**归档当前 `light_cci_grn+compare_N_kl` 的代码版本、运行配置、18 项 metrics 和 PIJ 输出，不覆盖现有最好结果。
2. **先完成原报告中的代码归属重构：**确保位置迁移前后当前结果数值回归通过。
3. **阶段一归因：**运行 Full、N-only、G-only、N+shuffled-G，回答当前结果由什么产生。
4. **形成决策：**只有 Full 明显优于公平 N-only 与 shuffled-G 后，才进入权重、维数、温度或更细的 feature 优化。

本轮停止边界：阶段一完成前，不增加 E/L/Sr/spatial，不重启 joint CCI-GRN，不验证纯 GRN 网络，不做 feature-related 下游相关性分析，也不以“把正 EI 数量继续调高”为目标展开无边界搜索。

一句话总结：当前只保留阶段一：解释“当前最好结果为什么好”。先把真实 GRN feature 的增量贡献与归一化、平滑和额外成本块效应区分开，再决定是否继续优化。

16. 阶段一的统一数学阅读框架

前面的新增章节给出了阶段一实验的研究目标、实验组和实现边界。本节开始只做一件事：把这些方案背后的数学过程完整展开，使每一个网络、特征、成本、转移概率和 EI 指标之间的关系都能够被逐步追踪。

理解整套实验时必须始终区分五个对象：

1. **网络矩阵：**描述同一时间点、同一层级内部，哪些 unit 之间存在边以及边有多强；
2. **网络特征：**把一个 unit 在网络中的结构角色压缩为一个有限维向量；
3. **跨时间成本：**衡量源时间点的一个 unit 与目标时间点的另一个 unit 有多不相似；
4. **PIJ 转移矩阵：**把跨时间成本转成逐行归一化的条件转移概率；
5. **EI 与 EI gain：**分别衡量微观层和宏观层的转移可辨识度，并比较粗粒化后是否获得宏观优势。

16.1 统一索引与维度约定

用 t_s 表示一个时间对中的**源时间点**，用 t_t 表示同一时间对中的**目标时间点**。下标 s 和 t 在这里分别来自 source 与 target，不代表具体的胚胎时间数值。例如，若实验时间对为 $11.5 \rightarrow 12.5$ ，则 $t_s = 11.5$ ， $t_t = 12.5$ 。

用 ℓ 表示层级，例如 spot、seurat_k150 或 seurat_k40。用

$$\mathcal{V}_{\ell,t} = \{v_1^{(\ell,t)}, \dots, v_{n_{\ell,t}}^{(\ell,t)}\}$$

表示层级 ℓ 在时间点 t 的 unit 集合。其中， $\mathcal{V}_{\ell,t}$ 是 unit 集合； $v_i^{(\ell,t)}$ 是其中第 i 个 unit； $n_{\ell,t} = |\mathcal{V}_{\ell,t}|$ 是该层级该时间点的 unit 数量；符号 $|\cdot|$ 表示集合元素个数。

对任一非 gene 层级，定义

$$A_{\ell,t}^{\text{CCI}} \in \mathbb{R}_{\geq 0}^{n_{\ell,t} \times n_{\ell,t}}$$

为 CCI 邻接矩阵。其中， $A_{\ell,t}^{\text{CCI}}$ 是矩阵； $\mathbb{R}_{\geq 0}$ 表示所有非负实数；矩阵第 i 行第 j 列元素 $A_{\ell,t,ij}^{\text{CCI}}$ 表示同一时间点、同一层级内从 unit i 指向 unit j 的通讯强度。矩阵是方阵，因为发送端和接收端都来自

同一组 unit。

后文用 d_N 表示 N 特征的维数，用 d_G 表示 G 特征的维数；用 $n_s = n_{\ell, t_s}$ 表示源时间点 unit 数，用 $n_t = n_{\ell, t_t}$ 表示目标时间点 unit 数。于是源特征矩阵和目标特征矩阵的一般形式分别为

$$F_s \in \mathbb{R}^{n_s \times d}, \quad F_t \in \mathbb{R}^{n_t \times d},$$

其中， F_s 和 F_t 是两个特征矩阵； d 是它们共同的特征维数；每一行对应一个 unit，每一列对应一个特征分量。源、目标的行数可以不同，但列数必须相同，因为跨时间距离需要逐特征比较。

16.2 从网络到 EI 的总流程

对任意一个层级 ℓ 和时间对 (t_s, t_t) ，当前 compare 系列方法可以抽象成以下链条：

$$(A_{\ell, t_s}, A_{\ell, t_t}) \xrightarrow{\text{网络表示}} (F_s, F_t) \xrightarrow{\text{跨时间成本}} C \xrightarrow{\exp(-C/\tau)} K \xrightarrow{\text{逐行归一化}} P \xrightarrow{\text{有效信息}} EI.$$

这里， A_{ℓ, t_s} 和 A_{ℓ, t_t} 是源、目标两个时间点的网络邻接矩阵； F_s 和 F_t 是从网络提取出的 unit 特征； $C \in \mathbb{R}_{\geq 0}^{n_s \times n_t}$ 是跨时间成本矩阵； $K \in \mathbb{R}_{\geq 0}^{n_s \times n_t}$ 是未经行归一化的相似性核； $\tau > 0$ 是 PIJ 温度； $P \in [0, 1]^{n_s \times n_t}$ 是最终 PIJ 转移矩阵； EI 是该转移矩阵的有效信息。

阶段一改变的是链条中的**特征成本来源**：比较 N、G、N+G 和 N+ 随机 G；网络邻接矩阵、NMF、KL、PIJ 与 EI 计算保持一致。

17. LightCCI 原始网络的完整原理

17.1 LightCCI 的基本定位：它首先是一个分层图构造器

LightCCI 的核心并不是直接给出跨时间 PIJ，而是为每一个时间点、每一个层级提供一个层内网络。它遵循如下分层规则：

- 在 gene 层，节点是基因，边来自 GRN；
- 在 spot 和各 domain 层，节点是 spot 或 domain，边来自对应层级的 CCI；
- 不把所有层级强行拼成一个巨型稠密图；
- 不在 network 文件中直接加入表达 PCA、NMF、KL 或 PIJ；这些操作由后续 PIJ 方法完成。

这一设计之所以称为“Light”，是因为 spot 层只保留现成 CCI 稀疏邻接，domain 层直接使用相应 domain 的 CCI，而不是在跨层级、跨时间的所有 unit 对之间构造高维 GRN-CCI 联合边。这样，内存复杂度主要取决于已有邻接矩阵的非零边数量，而不是所有可能 unit 对的平方。

17.2 非 gene 层：CCI 邻接矩阵如何形成

对层级 $\ell \neq \text{gene}$ 和时间点 t , COMMOT 可能直接给出总 CCI 矩阵, 也可能给出多张 ligand–receptor 矩阵。用 $L_{\ell,t}$ 表示该层级该时间点可用的 ligand–receptor 矩阵数量, 用

$$S_{\ell,t}^{(p)} \in \mathbb{R}^{n_{\ell,t} \times n_{\ell,t}}, \quad p = 1, \dots, L_{\ell,t},$$

表示第 p 张 ligand–receptor 通讯矩阵。其中, $S_{\ell,t,ij}^{(p)}$ 表示在第 p 个配体–受体对下, 从 unit i 到 unit j 的通讯分数。

如果总矩阵没有预先保存, LightCCI 首先计算

$$A_{\ell,t}^{\text{raw}} = \sum_{p=1}^{L_{\ell,t}} S_{\ell,t}^{(p)}.$$

这里, $A_{\ell,t}^{\text{raw}}$ 是尚未清洗的总通讯矩阵; 求和表示不同 ligand–receptor 通道对同一条 unit–unit 通讯边的贡献被累加。如果已经存在 `cci_total.npz`, 则该文件直接承担 $A_{\ell,t}^{\text{raw}}$ 的角色, 不重复求和。

随后, 代码按照 CCI index 文件给出的 unit 顺序对矩阵行列进行对齐, 并执行非负清洗。用 $\eta \geq 0$ 表示配置中的 `cci_min` 阈值, 定义逐元素清洗算子

$$\mathcal{T}_{\eta}(z) = \begin{cases} 0, & z < \eta \text{ 或 } z \text{ 不是有限数;} \\ z, & z \geq \eta \text{ 且 } z \text{ 为有限非负数.} \end{cases}$$

这里, z 表示任意一个原始边权; $\mathcal{T}_{\eta}$ 表示清洗和阈值化操作; 有限数是指不是 NaN、正无穷或负无穷的数。最终邻接为

$$A_{\ell,t,ij}^{\text{CCI}} = \mathcal{T}_{\eta}(A_{\ell,t,ij}^{\text{raw}}).$$

这个公式的目的不是创造新通讯边, 而是保留原有 CCI 分数中可用的非负部分。负值在这里没有明确的通讯强度语义, 因此被截为零; 低于 η 的边可以被删除, 以避免极小噪声值占据稀疏矩阵。需要特别注意: 代码中为 edge table 计算的 min–max 值主要用于记录和导出, 真正存入 graph metadata 并供 N 特征使用的是清洗后的原始 CCI 邻接 $A_{\ell,t}^{\text{CCI}}$ 。

17.3 gene 层：GRN 邻接矩阵如何形成

在 gene 层，unit 本身就是基因。用 G_t 表示时间点 t 保留下来的基因数量，用

$$\omega_{t,g \rightarrow h} \in \mathbb{R}$$

表示 GRN 文件中从调控基因 g 指向靶基因 h 的原始权重。这里， g 和 h 都是基因索引；箭头 $g \rightarrow h$ 表示有向调控关系。

LightCCI gene 层使用绝对强度构造邻接：

$$A_{t,gh}^{\text{GRN}} = |\omega_{t,g \rightarrow h}|, \quad A_t^{\text{GRN}} \in \mathbb{R}_{\geq 0}^{G_t \times G_t}.$$

其中， $|\omega|$ 表示绝对值。这样做的原因是后续非负矩阵分解要求输入非负，而当前实验首先研究调控连接强弱，不在这个阶段区分激活与抑制的符号。该选择会丢失正负调控方向，因此它是当前方法的建模边界，而不是对 GRN 全部生物语义的完整表达。

17.4 为什么 LightCCI 本身还不是 PIJ

在 `light_cci.py` 中，`lower_mats` 和 `upper_mats` 被设置为空特征矩阵，真正的邻接保存在各个 `LayerGraph.metadata["adjacency_csr"]` 中。这意味着 LightCCI 只回答：

在时间点 t 和层级 ℓ ，同层 unit 之间的网络连接是什么？

它还没有回答：

源时间点的 unit i 应当以多大概率转移到目标时间点的 unit j ？

后一个问题需要先从邻接中提取可比较的 unit 表示，再构造跨时间成本。当前 `compare_N_kl` 使用的 N 特征正是完成这一桥接的结构表示。

18. N 特征、KL 成本、PIJ 与 EI 的数学过程

18.1 为什么 N 特征按时间对构造

对每个时间对 (t_s, t_t) ， N 特征只同时查看这两个时间点的邻接，不把其他时间点共同放进同一次分解。这样得到的是 pairwise 表示：

$$(A_{\ell,t_s}, A_{\ell,t_t}) \longrightarrow (N_s, N_t).$$

其中， $N_s \in \mathbb{R}^{n_s \times d_N}$ 是源时间点 N 特征； $N_t \in \mathbb{R}^{n_t \times d_N}$ 是目标时间点 N 特征；二者特征维数 d_N 相同。

按时间对分解的目的，是让源、目标两个时间点共享同一组潜在因子语义，同时避免利用时间对之外的数据。它不是在每个时间点各自随意做一次 NMF 后直接比较，因为两次独立 NMF 的第 k 个分量不保证具有相同含义。

18.2 spot 层：共享核心的有向 NMF

spot 层的 CCI 是有向图，而且不同时间点的 spot 数量可以不同。因此，当前代码使用共享核心有向 NMF。令

$$A_s = A_{\text{spot},t_s}^{\text{CCI}} \in \mathbb{R}_{\geq 0}^{n_s \times n_s}, \quad A_t = A_{\text{spot},t_t}^{\text{CCI}} \in \mathbb{R}_{\geq 0}^{n_t \times n_t}.$$

这里， A_s 和 A_t 分别是源、目标时间点的 spot CCI 邻接。

给定潜在秩 $r \geq 1$ ，分解寻找五个非负矩阵：

$$U_s, V_s \in \mathbb{R}_{\geq 0}^{n_s \times r}, \quad U_t, V_t \in \mathbb{R}_{\geq 0}^{n_t \times r}, \quad B \in \mathbb{R}_{\geq 0}^{r \times r}.$$

其中， U_s 和 U_t 表示源、目标 spot 的发送端潜在角色； V_s 和 V_t 表示源、目标 spot 的接收端潜在角色； B 是两个时间点共享的角色交互核心； r 是 NMF 分量数，当前默认 `nmf_components=5`。

目标是近似：

$$A_s \approx U_s B V_s^T, \quad A_t \approx U_t B V_t^T.$$

上标 T 表示矩阵转置。完整优化目标可写成

$$\min_{U_s, V_s, U_t, V_t, B \geq 0} \|A_s - U_s B V_s^T\|_F^2 + \|A_t - U_t B V_t^T\|_F^2.$$

这里， $\|M\|_F$ 表示矩阵 M 的 Frobenius 范数，即所有元素平方和再开平方；约束“ ≥ 0 ”表示所有因子逐元素非负。

为什么需要共享 B ？如果两个时间点分别使用完全独立的核心，那么源时间点的潜在角色 1 和目标时间点的潜在角色 1 不一定对应同一种通讯模式。共享 B 强制两个时间点用同一套“发送角色如何连接接收角色”的潜在语法，而 U 、 V 允许每个 spot 在这套语法中的参与程度随时间变化。

spot 的最终 N 特征是发送角色与接收角色的拼接：

$$N_s = [U_s \mid V_s], \quad N_t = [U_t \mid V_t].$$

符号 $[\cdot \mid \cdot]$ 表示按列拼接。因此

$$N_s \in \mathbb{R}^{n_s \times 2r}, \quad N_t \in \mathbb{R}^{n_t \times 2r}.$$

当默认 $r = 5$ 时，spot 的 N 特征维数 $d_N = 2r = 10$ 。拼接而不是相加，是为了保留一个 spot “擅长向外发送”与“擅长从外部接收”这两种不同的有向角色。

18.3 非 spot 的固定节点层：普通 pairwise joint NMF

对 unit 集合和列空间能够对齐的非 spot 层，代码使用共享右因子的 pairwise joint NMF。令源、目标邻接仍记为 A_s 和 A_t ，并要求它们列数相同。给定秩 r ，寻找

$$W_s \in \mathbb{R}_{\geq 0}^{n_s \times r}, \quad W_t \in \mathbb{R}_{\geq 0}^{n_t \times r}, \quad H \in \mathbb{R}_{\geq 0}^{r \times m}.$$

其中， m 是两个邻接矩阵共同的列数； W_s 和 W_t 是 unit 的低维系数； H 是两个时间点共享的网络基底。

目标近似为

$$A_s \approx W_s H, \quad A_t \approx W_t H,$$

对应优化目标

$$\min_{W_s, W_t, H \geq 0} \|A_s - W_s H\|_F^2 + \|A_t - W_t H\|_F^2.$$

最终定义

$$N_s = W_s, \quad N_t = W_t,$$

所以此时 $d_N = r$ ，默认是 5。共享 H 的目的与共享 B 类似：让源、目标两个时间点的 N 分量处在同一坐标系中，使第 k 列可以跨时间比较。

18.4 N 特征为什么还要做 pairwise z-score

NMF 各分量的绝对尺度可能不同。为了避免某一列仅因为数值范围更大就在 KL softmax 中占据主导，代码把同一时间对、同一 side 的源与目标特征拼在一起估计均值和标准差。

对特征列 $k = 1, \dots, d_N$ ，定义

$$\mu_k = \frac{1}{n_s + n_t} \left(\sum_{i=1}^{n_s} N_{s,ik} + \sum_{j=1}^{n_t} N_{t,jk} \right),$$

其中， μ_k 是第 k 个 N 分量在源、目标合并样本上的均值； $N_{s,ik}$ 是源 unit i 的第 k 个分量； $N_{t,jk}$ 是目标 unit j 的第 k 个分量。

再定义标准差

$$\sigma_k = \sqrt{\frac{1}{n_s + n_t} \left[\sum_{i=1}^{n_s} (N_{s,ik} - \mu_k)^2 + \sum_{j=1}^{n_t} (N_{t,jk} - \mu_k)^2 \right]}.$$

这里， σ_k 衡量第 k 列在该时间对中的波动。标准化后的特征为

$$Z_{s,ik}^N = \frac{N_{s,ik} - \mu_k}{\sigma_k}, \quad Z_{t,jk}^N = \frac{N_{t,jk} - \mu_k}{\sigma_k}.$$

其中， Z_s^N 和 Z_t^N 是进入 KL 的标准化 N 特征。如果 $\sigma_k = 0$ ，代码把该列输出为零，避免除零。源

和目标共同拟合 μ_k, σ_k ，是为了保证二者使用同一尺度；如果分别标准化，两个时间点的绝对相对位置会被人为抹掉。

18.5 把特征向量转为概率分布

KL 散度要求输入是概率分布，而标准化特征可以为负，也不保证元素和为 1。因此，对任一源 unit i ，将其标准化 N 特征通过行 softmax 转为

$$\pi_{s,ik}^N = \frac{\exp(Z_{s,ik}^N/\beta)}{\sum_{h=1}^{d_N} \exp(Z_{s,ih}^N/\beta)}, \quad k = 1, \dots, d_N.$$

这里， $\pi_{s,ik}^N$ 是源 unit i 在第 k 个潜在网络分量上的概率质量； h 是 softmax 分母中的求和索引； $\exp$ 是自然指数函数； $\beta > 0$ 是 feature softmax 温度，当前代码从 `pj_entropy_epsilon` 读取，默认值为 0.05。

目标 unit j 的分布同理：

$$\pi_{t,jk}^N = \frac{\exp(Z_{t,jk}^N/\beta)}{\sum_{h=1}^{d_N} \exp(Z_{t,jh}^N/\beta)}.$$

β 越小，softmax 越强调最大的少数分量，分布越尖锐； β 越大，分布越平坦。当前阶段必须固定 β ，因为改变它会同时改变所有 KL 成本的几何结构，不能与是否加入 GRN 混在同一次归因实验中。

18.6 源到目标的 KL 成本

对源 unit i 与目标 unit j ， N block 的 KL 成本定义为

$$C_{ij}^N = D_{\text{KL}}(\pi_{s,i}^N \parallel \pi_{t,j}^N) = \sum_{k=1}^{d_N} \pi_{s,ik}^N \log \frac{\pi_{s,ik}^N}{\pi_{t,jk}^N}.$$

其中， $C_{ij}^N \geq 0$ 是从源 unit i 到目标 unit j 的结构特征不匹配成本； $D_{\text{KL}}(p \parallel q)$ 表示从概率分布 p 到概率分布 q 的 KL 散度； $\log$ 是自然对数； $\pi_{s,i}^N$ 和 $\pi_{t,j}^N$ 分别表示对应的整行概率向量。

这个成本是有方向的，一般有

$$D_{\text{KL}}(p \parallel q) \neq D_{\text{KL}}(q \parallel p).$$

这与 PIJ 的源到目标方向一致：我们关心的是“源 unit 的潜在网络角色用目标 unit 的角色分布来近似时损失多少”，而不是构造无方向的几何距离。代码用一个很小的正数 $\epsilon_{\text{KL}} = 10^{-12}$ 裁剪概率，防止出现 $\log 0$ ；这里的 ϵ_{KL} 与上面的 softmax 温度 β 不是同一个量。

18.7 从成本到 PIJ

给定任意非负成本矩阵 $C \in \mathbb{R}_{\geq 0}^{n_s \times n_t}$ ，先定义未归一化核

$$K_{ij} = \exp\left(-\frac{C_{ij}}{\tau}\right).$$

其中， $K_{ij} > 0$ 是源 unit i 与目标 unit j 的相似性权重； $\tau > 0$ 是 PIJ 温度，当前默认 `pij_temperature=1.0`。成本越小， K_{ij} 越接近 1；成本越大， K_{ij} 越接近 0。

随后对每一行归一化：

$$P_{ij} = \frac{K_{ij}}{\sum_{j'=1}^{n_t} K_{ij'}}.$$

其中， j' 是目标 unit 的求和索引； P_{ij} 表示在源 unit 固定为 i 时转移到目标 unit j 的条件概率；每一行满足

$$\sum_{j=1}^{n_t} P_{ij} = 1.$$

τ 越小，成本差异被放大，每行 PIJ 更集中； τ 越大，每行更接近均匀。阶段一固定 τ ，因为温度变化本身就可能显著改变 EI。

18.8 EI 与 EI gain 的代码定义

代码默认对源 unit 做均匀干预，即

$$p_I(i) = \frac{1}{n_s}, \quad i = 1, \dots, n_s.$$

其中，随机变量 I 表示源 unit； $p_I(i)$ 是干预选择源 unit i 的概率。均匀干预表示每一个源状态被赋予相同权重，不让样本频率决定 EI。

目标边际分布为

$$p_J(j) = \frac{1}{n_s} \sum_{i=1}^{n_s} P_{ij}, \quad j = 1, \dots, n_t.$$

其中，随机变量 J 表示目标 unit； $p_J(j)$ 是在均匀源干预下到达目标 unit j 的总体概率。

有效信息定义为互信息

$$EI(P) = I(I; J) = \frac{1}{n_s} \sum_{i=1}^{n_s} \sum_{j=1}^{n_t} P_{ij} \log_2 \frac{P_{ij}}{p_J(j)}.$$

这里， $I(I; J)$ 是源随机变量和目标随机变量之间的互信息； $\log_2$ 是以 2 为底的对数，因此 EI 单位是 bit。

代码使用的等价熵形式为

$$EI(P) = H(J) - H(J | I),$$

其中

$$H(J) = - \sum_{j=1}^{n_t} p_J(j) \log_2 p_J(j)$$

是目标边际熵,

$$H(J | I) = - \frac{1}{n_s} \sum_{i=1}^{n_s} \sum_{j=1}^{n_t} P_{ij} \log_2 P_{ij}$$

是给定源状态后的平均目标条件熵。

直观上, $H(J)$ 大表示总体上可以到达多个不同目标; $H(J | I)$ 小表示知道源 unit 后, 目标分布更加确定。二者相减得到“源状态能够区分目标结果多少”。如果每一行 PIJ 完全相同, 知道源 unit 也不能改变对目标的预测, 此时 EI 接近 0。

对同一个时间对, 设微观层 PIJ 为 P^{lower} , 宏观层 PIJ 为 P^{upper} , 则

$$EI_{\text{lower}} = EI(P^{\text{lower}}), \quad EI_{\text{upper}} = EI(P^{\text{upper}}),$$

并定义

$$EI_{\text{gain}} = EI_{\text{upper}} - EI_{\text{lower}}.$$

其中, $EI_{\text{gain}} > 0$ 表示宏观层的有效信息高于微观层; $EI_{\text{gain}} < 0$ 表示粗粒化后有效信息下降。阶段一的目标不是直接修改 EI 公式, 而是在固定网络的条件下改变进入 PIJ 的 feature 成本来源, 观察 EI_{gain} 是否发生稳定、可归因的变化。

19. 当前 LightCCI-GRN 与 N/G block-KL 的完整原理

19.1 GRN 输入的对齐、截断与行归一化

对某一非 gene 层级、某一时间点, 设该层共有 n 个 unit, 保留下来的 GRN 基因数为 G 。定义表达矩阵

$$X \in \mathbb{R}_{\geq 0}^{n \times G},$$

其中, X_{ig} 表示 unit i 中基因 g 的非负表达量; 矩阵第 i 行记为行向量 $x_i \in \mathbb{R}_{\geq 0}^G$ 。

原始 GRN 边表中, 从基因 g 到基因 h 的权重记为 $\omega_{g \rightarrow h}$ 。当前代码先取绝对值, 只保留表达矩阵中同时存在的 regulator 和 target, 并对每个 regulator 按权重从大到小最多保留 K 个 target。这里, K 是 `grn_topk_targets`, 当前默认 $K = 50$ 。

对保留下来的边, 定义行归一化 GRN 邻接

$$W_{gh} = \begin{cases} \frac{|\omega_{g \rightarrow h}|}{\sum_{h'=1}^G |\omega_{g \rightarrow h'}|}, & \sum_{h'=1}^G |\omega_{g \rightarrow h'}| > 0, \\ 0, & \text{否则.} \end{cases}$$

其中, $W \in \mathbb{R}_{\geq 0}^{G \times G}$ 是行归一化后的 GRN; W_{gh} 表示 regulator g 分配给 target h 的相对权重; h' 是对

同一 regulator 的所有 target 求和时使用的索引。

为什么要按行归一化？不同 regulator 的原始总权重可能相差很大。行归一化把每个有出边的 regulator 的总权重缩放为 1，使后续 Wx_i 更接近“该 regulator 的 target 表达加权平均”，而不是让拥有大量高权重边的 regulator 仅凭网络度数支配 GRN state。

19.2 双端表达门控的两个向量

对 unit i ，先定义 regulator program

$$p_i^{\text{reg}} = Wx_i^{\text{T}} \in \mathbb{R}_{\geq 0}^G.$$

这里， x_i^{T} 是把行向量 x_i 转成列向量； $p_{i,g}^{\text{reg}} = \sum_h W_{gh}x_{i,h}$ 表示 regulator g 所连接的 targets 在 unit i 中有多活跃。

随后定义发送端或 regulator-side state

$$R_i = x_i \odot p_i^{\text{reg}} = x_i \odot (Wx_i^{\text{T}}).$$

其中， $R_i \in \mathbb{R}_{\geq 0}^G$ ；符号 $\odot$ 表示逐元素乘法； $R_{i,g} = x_{i,g} p_{i,g}^{\text{reg}}$ 。这个乘法要求两个条件同时满足：regulator 基因 g 自身在 unit i 中表达，而且它所指向的 targets 也整体活跃。因此， R_i 不是单纯的表达，也不是单纯的 GRN 度数，而是表达状态经过 GRN 出边结构门控后的发送调控状态。

接着定义 target program

$$p_i^{\text{tar}} = W^{\text{T}}x_i^{\text{T}} \in \mathbb{R}_{\geq 0}^G.$$

其中， $p_{i,h}^{\text{tar}} = \sum_g W_{gh}x_{i,g}$ 表示指向 target h 的 regulators 在 unit i 中有多活跃。

接收端或 target-side state 为

$$T_i = x_i \odot p_i^{\text{tar}} = x_i \odot (W^{\text{T}}x_i^{\text{T}}).$$

其中， $T_i \in \mathbb{R}_{\geq 0}^G$ ； $T_{i,h} = x_{i,h} p_{i,h}^{\text{tar}}$ 。它要求 target 基因 h 自身表达，同时其上游 regulators 也活跃，因此描述 unit 对调控输入的接收或响应状态。

双端设计的理由是 CCI 本身具有发送和接收方向。只使用 R_i 会忽略 unit 作为受体的状态，只使用 T_i 会忽略 unit 作为发送者的调控输出。当前 state 同时保留二者，再在投影后合并。

19.3 确定性随机投影为什么存在

R_i 和 T_i 都有 G 维, G 可能达到数千。若直接在所有源-目标 unit 对上计算高维 KL, 内存和计算成本都会显著增加。当前代码为 regulator 和 target 两个角色分别构造投影矩阵:

$$Q^{\text{reg}}, Q^{\text{tar}} \in \mathbb{R}^{G \times d_G}.$$

其中, d_G 是 grn_state_dim, 当前默认 $d_G = 64$; Q^{reg} 是 regulator 角色投影; Q^{tar} 是 target 角色投影。

对基因 g 和投影维度 k , 投影元素服从近似

$$Q_{gk}^{\text{role}} \sim \mathcal{N}\left(0, \frac{1}{d_G}\right),$$

其中, $\mathcal{N}(0, 1/d_G)$ 表示均值为 0、方差为 $1/d_G$ 的高斯分布; $\text{role} \in \{\text{reg}, \text{tar}\}$ 表示角色。实际实现不是每次运行随意抽样, 而是把 projection seed、角色字符串和基因名输入 SHA-256 哈希, 再由哈希确定该基因该角色的随机行。因此, 只要基因名、角色、seed 和 d_G 不变, 投影矩阵就完全可复现。

unit i 的投影 GRN feature 定义为

$$G_i = R_i Q^{\text{reg}} + T_i Q^{\text{tar}} \in \mathbb{R}^{d_G}.$$

这里, G_i 是当前 block-KL 使用的 G feature; $R_i Q^{\text{reg}}$ 是发送调控状态的低维投影; $T_i Q^{\text{tar}}$ 是接收调控状态的低维投影。

为什么 regulator 与 target 使用不同投影矩阵? 如果使用同一矩阵, 同一个基因在发送角色和接收角色中会落到完全相同的随机方向, 两类角色更容易混淆。不同的 role-specific projection 让相同基因在两种角色中拥有不同编码。为什么最终相加而不是保留 $2d_G$ 维拼接? 当前实现选择相加是为了把维数固定在 $d_G = 64$, 降低成本; 这是一项当前算法选择, 阶段一归因实验不改变它。

对整个层级, 把所有 unit 的 G_i 按行堆叠得到

$$G \in \mathbb{R}^{n \times d_G}.$$

与 N 特征一样, 对每一个时间对和每一个 lower/upper side, 源 G 和目标 G 会共同做 pairwise z-score。标准化公式与前文 N 特征完全相同, 只需把 N 替换为 G 。

19.4 G block 的 KL 成本

标准化后的源、目标 G 特征分别记为 $Z_s^G \in \mathbb{R}^{n_s \times d_G}$ 和 $Z_t^G \in \mathbb{R}^{n_t \times d_G}$ 。使用与 N block 相同的 softmax 温度 β ，定义

$$\pi_{s,ik}^G = \frac{\exp(Z_{s,ik}^G/\beta)}{\sum_{h=1}^{d_G} \exp(Z_{s,ih}^G/\beta)}, \quad \pi_{t,jk}^G = \frac{\exp(Z_{t,jk}^G/\beta)}{\sum_{h=1}^{d_G} \exp(Z_{t,jh}^G/\beta)}.$$

这里， $k, h = 1, \dots, d_G$ 是 G feature 维度索引； $\pi_{s,i}^G$ 和 $\pi_{t,j}^G$ 是源、目标 unit 的 G feature 概率表示。

G block KL 成本为

$$C_{ij}^G = \sum_{k=1}^{d_G} \pi_{s,ik}^G \log \frac{\pi_{s,ik}^G}{\pi_{t,jk}^G}.$$

其中， $C^G \in \mathbb{R}_{\geq 0}^{n_s \times n_t}$ ，形状与 C^N 完全相同，所以两个成本可以逐元素融合。

19.5 为什么两个成本要独立稳健归一化

N block 和 G block 的维数、softmax 尖锐度和 KL 数值范围可能不同。若直接计算

$$w_N C^N + w_G C^G,$$

即使 $w_N = w_G$ ，数值范围较大的 block 也会实际占据主导。因此当前完整方法先对每个成本独立做 5%–95% 稳健归一化。

对任意成本矩阵 C ，把所有有限元素的第 5 百分位数和第 95 百分位数分别记为

$$q_{0.05}(C), \quad q_{0.95}(C).$$

其中， $q_a(C)$ 表示矩阵有限元素的 a 分位数。定义

$$\tilde{C}_{ij} = \text{clip}\left(\frac{C_{ij} - q_{0.05}(C)}{q_{0.95}(C) - q_{0.05}(C)}, 0, 1\right).$$

这里， $\tilde{C}$ 是归一化成本； $\text{clip}(z, 0, 1)$ 表示小于 0 的值设为 0，大于 1 的值设为 1，其余保持不变。如果两个分位数相同，代码退回 min–max；如果连 min–max 也退化，则输出零矩阵。

这一操作的目的是，让 N 和 G 各自在典型成本范围内映射到 $[0, 1]$ ，降低极端离群值的影响。代价是它会丢失两个 block 的绝对尺度信息，因此阶段一必须用严格对照判断结果是否来自真实 G，而不是仅来自归一化带来的成本重标定。

19.6 当前完整 block-KL 成本

用 $w_N \geq 0$ 和 $w_G \geq 0$ 分别表示 N 与 G 的权重，并要求

$$w_N + w_G = 1.$$

当前默认

$$w_N = 0.5, \quad w_G = 0.5.$$

完整成本为

$$C_{ij}^{\text{full}} = w_N \tilde{C}_{ij}^N + w_G \tilde{C}_{ij}^G.$$

其中， $\tilde{C}^N$ 和 $\tilde{C}^G$ 是独立稳健归一化后的两个成本。该公式是凸加权平均，因为权重非负且和为 1，所以 $C_{ij}^{\text{full}} \in [0, 1]$ 。

为什么使用加权平均？它表达的是“一个候选跨时间对应同时接受 CCI 网络结构证据和 GRN 状态证据的评价”。当两者都小，组合成本小；当其中一个明显不匹配，组合成本上升。当前使用 0.5/0.5 并不代表已经证明两个信息源同等重要，而是一个对称起点；阶段一首先检验真实 G 是否有贡献，不立即搜索最优权重。

组合成本随后仍通过

$$K_{ij} = \exp(-C_{ij}^{\text{full}}/\tau), \quad P_{ij} = K_{ij} / \sum_{j'} K_{ij'}$$

得到 PIJ，再按前文相同公式计算 EI 和 EI gain。

20. 阶段一四组实验的变量控制与因果解释

20.1 阶段一究竟要估计什么

阶段一的研究对象不是“哪个超参数能让结果最好”，而是当前固定算法内**真实 GRN unit 对应关系的增量作用**。可以把每一个固定的器官、层级对、时间对组合记为一个任务 e 。对任务 e ，定义

$$Y_e(m) = EI_{\text{gain},e}^{(m)},$$

其中， m 表示实验模式； $Y_e(m)$ 是模式 m 在同一任务 e 上得到的 EI gain。

阶段一最核心的配对差值是

$$\Delta_e^{\text{realG}} = Y_e(\text{full}) - Y_e(\text{N-only}).$$

这里， Δ_e^{realG} 表示在相同 N、相同归一化和相同 PIJ 条件下加入真实 G 后的增量。因为两个结果来自完全相同的任务 e ，应进行逐任务配对比较，而不是只比较两个互不对应的总体均值。

另一个关键差值是

$$\Delta_e^{\text{identity}} = Y_e(\text{full}) - \frac{1}{S} \sum_{a=1}^S Y_e(\text{shuffle}, a).$$

其中， S 是 shuffle seed 数，第一轮要求 $S \geq 5$ ； a 是 seed 索引； $Y_e(\text{shuffle}, a)$ 是第 a 个随机置换对照的 EI gain。 $\Delta_e^{\text{identity}}$ 衡量真实 unit-GRN 对应相对于仅保留 G 数值分布的随机对应是否更好。

20.2 Full：当前结果作为冻结锚点

Full 组使用

$$C^{\text{full}} = 0.5\tilde{C}^N + 0.5\tilde{C}^G.$$

这一组必须使用当前相同的 LightCCl-GRN network、NMF、GRN state、投影、z-score、KL、稳健归一化、温度和导出路径。它的作用是作为已经观察到最好结果的 anchor，而不是在阶段一中重新调参。

20.3 N-only：只关闭 G，但保留 Full 的成本尺度

公平 N-only 定义为

$$C^{\text{N-only}} = \tilde{C}^N.$$

它与当前代码中 `weight_g=0` 的历史 exact fallback 不完全相同。历史 fallback 返回未经稳健归一化的 C^N ，而公平 N-only 返回 $\tilde{C}^N$ 。如果用历史 fallback 与 Full 比较，那么同时改变了“是否使用 G”和“N 成本是否被归一化”两个变量，无法把差异归因于 G。

因此需要保留历史 fallback 以保证旧命令兼容，同时新增显式 `n_only_control`：它使用相同的 C^N 、相同的 5%-95% 归一化、相同的 τ ，唯一关闭的是 G block。

20.4 G-only: 判断 G 是独立信号还是互补信号

公平 G-only 定义为

$$C^{G\text{-only}} = \tilde{C}^G.$$

该实验不要求 G-only 一定优于 Full。它回答的是：只根据 GRN state，是否已经能够建立有效的跨时间转移？

若 G-only 好，说明 G 本身包含较强的跨时间状态信息；若 G-only 差但 Full 好，说明 G 更可能补充 N 所缺失的信息，而不是独立替代 CCI 网络结构。这种“互补而非替代”的结果完全符合当前研究问题，不应被视为失败。

20.5 N+shuffled-G: 只破坏 unit 身份对应

设标准化后的真实源 G 特征为 $Z_s^G \in \mathbb{R}^{n_s \times d_G}$ ，真实目标 G 特征为 $Z_t^G \in \mathbb{R}^{n_t \times d_G}$ 。对一个固定 seed a ，构造两个置换矩阵

$$\Pi_s^{(a)} \in \{0, 1\}^{n_s \times n_s}, \quad \Pi_t^{(a)} \in \{0, 1\}^{n_t \times n_t}.$$

置换矩阵的每一行和每一列恰好有一个 1，其余为 0；左乘置换矩阵等价于重新排列 feature 行。

定义

$$Z_s^{G, \text{shuf}(a)} = \Pi_s^{(a)} Z_s^G, \quad Z_t^{G, \text{shuf}(a)} = \Pi_t^{(a)} Z_t^G.$$

其中， $Z_s^{G, \text{shuf}(a)}$ 和 $Z_t^{G, \text{shuf}(a)}$ 保留真实 G 的全部行向量和数值分布，但这些行不再属于原来的 unit。源、目标分别在各自的时间点、层级和 side 内置换，不跨器官、不跨层级、不跨时间混合。

由置换后的 G 计算 $C^{G, \text{shuf}(a)}$ ，并定义

$$C^{\text{shuffle}(a)} = 0.5\tilde{C}^N + 0.5\tilde{C}^{G, \text{shuf}(a)}.$$

这一对照与 Full 使用相同维数、相同 G 行向量集合、相同 KL、相同稳健归一化和相同权重，唯一破坏的是“哪个 unit 拥有哪个 GRN state”。因此，Full 优于 shuffle 才能支持 unit-specific 的真实 GRN 信息有作用。

20.6 四组实验中必须保持不变的变量

变量	固定要求及原因
LightCCI CCI 邻接	四组使用完全相同的 network payload 和 unit 顺序，不能在阶段一同时修改 CCI 边。
时间对与层级对	四组运行完全相同的任务集合，所有结论按任务配对比较。
NMF 类型、秩与 seed	spot 仍使用共享核心有向 NMF，其他层仍使用 pairwise joint NMF； r 、迭代次数和 seed 不变。

变量	固定要求及原因
GRN 边预处理	真实 G、G-only 与 shuffle 都使用相同基因对齐、绝对权重、每 regulator top- K 和行归一化。
GRN 投影	d_G 、projection seed、gene-role 哈希和 regulator/target 相加规则不变。
pairwise z-score	N 与 G 都按当前规则在每个时间对、每个 side 上由源和目标共同拟合。
KL softmax 温度	β 固定，不能一边做消融一边改变概率分布尖锐程度。
成本稳健归一化	所有公平控制组都调用与 Full 相同的 5%-95% 归一化；只保留旧 fallback 作为兼容路径，不把它当公平对照。
PIJ 温度	τ 固定，避免将 PIJ 平滑程度变化误认为 G 的作用。
Full 与 shuffle 权重	两组都固定 $w_N = w_G = 0.5$ ；否则随机对照不再只差 unit 身份。
导出与 EI 公式	使用同一 PIJ 行归一化、同一 EI 代码和同一数值精度。

20.7 为什么这四组已经足够回答第一轮问题

四组形成一个最小逻辑闭环：

1. N-only 给出在相同成本尺度下没有 G 的结果；
2. G-only 判断 G 是否可以独立工作；
3. Full 给出 N 与真实 G 的联合结果；
4. shuffle 判断联合结果是否依赖真实 unit-GRN 对应，而不是仅仅多加了一块随机成本。

阶段一不需要同时增加更多 feature，因为增加 E、L、Sr 会产生新的信息源，使“Full 为什么好”再次无法归因。它也不需要先扫描 w_G ，因为权重搜索回答的是“什么权重最好”，而不是“真实 G 是否有用”。

21. 阶段一公式设计的取舍与边界

21.1 为什么阶段一不改变网络

阶段一的目标是解释当前最好结果。如果同时修改 CCI 邻接，就无法知道结果变化来自 feature block 还是网络。因此，阶段一把 A^{CCI} 固定，只改变 G block 的存在、独立性和 unit 身份。

21.2 为什么第一轮不扫描多个权重、 d_G 、 β 或 τ

w_N 与 w_G 控制 N/G 两个成本块的相对权重， d_G 控制 G feature 压缩维数， β 控制特征 softmax 尖锐度， τ 控制 PIJ 平滑度。这些量都能改变结果。如果第一轮同时搜索它们，当前最好结果可能变成多重选择后的最优点，难以判断真实 GRN 是否具有稳健作用。因此第一轮冻结当前 Full 的默认值，只完成公平 N-only、G-only 和 shuffled-G 对照。

21.3 为什么随机对照必须保留分布

完全生成一组任意高斯噪声会同时改变 G 的均值、方差、维数和相关结构。unit 置换则保留每一条真实 feature 行，只破坏其生物身份。因此它更接近“除真实对应关系外其他条件都相同”的负对照。

21.4 当前结论能够说到哪里

完成阶段一后，若 Full 稳定优于 N-only 和 shuffled-G，可以说：

在固定 LightCCl 图和固定 PIJ 构造下，真实 unit-level GRN feature 为因果涌现结果提供了可归因的增量信息。

该结论不等价于“所有原始细胞通讯网络天然都存在因果涌现”，也不等价于“GRN 单独已经被证明存在因果涌现”。当前阶段只研究：在一个明确、可计算的 LightCCl 框架内，真实 GRN feature 是否对观测到的因果涌现提供可归因的增量信息。

22. 与代码重构后的文件归属对应关系

前述原报告已经规定 `grn_state.py` 并回 `light_cci_grn.py`, `block_kl.py` 并回 `compare_N_kl.py`。阶段一新增实现必须遵守这一归属边界。

22.1 阶段一实现位置

阶段一的 Full、N-only、G-only 和 shuffle-G 都属于 `compare_N_kl` 的私有实验行为。因此：

- 模式分支、成本选择、shuffle 和 block diagnostics 放在 `compare_N_kl.py`;
- 默认模式仍为 Full，旧命令和当前最好结果不得变化;
- 公平 N-only 必须显式调用和 Full 相同的成本归一化;
- shuffle seed、置换范围和模式写入配置与输出 metadata;
- 不为四组实验重新建立游离的 `ablation.py` 或 `block_kl.py`。

22.2 最低数学回归测试

除原报告中的位置迁移回归外，新实验至少增加以下数学测试：

1. 阶段一 Full 模式与当前最好结果 bitwise 或数值容差一致；
2. 公平 N-only 的成本等于 $\tilde{C}^N$ ，而不是历史 raw C^N ；
3. G 行置换前后每列均值、标准差和全部行向量多重集合不变；
4. G-only 仅使用 $\tilde{C}^G$ ，且不触发 N block；
5. 四种模式进入 PIJ 后使用相同的候选边、 β 、 τ 与裁剪规则。

最终理解：LightCCI 负责提供同层网络；NMF 把网络压缩成跨时间可比较的 N 特征；KL 把 unit 特征差异变成成本；指数核与行归一化把成本变成 PIJ；EI 衡量源状态对目标状态的可辨识信息。当前只保留阶段一，用公平 N-only、G-only 与 shuffled-G 对照检验真实 G feature 是否为当前最好结果提供可归因增量。

23. 当前主方法的完整原理：LightCCI-GRN + N + KL

本章专门把当前最好方法从头到尾连成一条完整链条。前文已经分别解释过 LightCCI、GRN state、N 特征和 block-KL，本章不再把它们当成互不相关的零件，而是回答以下核心问题：

当命令中使用 `network-methodlight_cci_grn` 和 `pj-methodcompare_N_kl` 时，程序究竟读取了什么数据、怎样构造每个 unit 的两类特征、怎样计算跨时间对应成本、怎样把成本变成 PIJ，又怎样从 PIJ 得到上下层 EI 与 EI gain？

这套方法可以先用一句话概括：

LightCCI 提供每个时间点、每个层级内部的 CCI 邻接；NMF 把 CCI 邻接压缩成网络结构特征 N；同一批 unit 的表达与 GRN 被转化成调控状态特征 G；N 和 G 分别通过 KL 散度形成两个跨时间成本矩阵；两个成本经过独立稳健归一化后加权融合；融合成本再通过指数核和逐行归一化变成 PIJ；最后分别计算 lower 层和 upper 层的 EI，并以二者之差作为 EI gain。

23.1 先澄清方法名称：名称中只有 N，但实际完整路径包含 G

注册方法名 `compare_N_kl` 的静态声明仍然是“使用 N 特征和 KL 距离”。但是，当 `network` 是 `light_cci_grn` 时，`network graph` 的 `metadata` 中额外保存了 GRN state。特征构造代码发现这个 payload 后，会为每个时间对生成 G block；KL 路径发现 G block 可用后，会自动进入 N/G block-KL。

因此，当前完整方法更准确的数学名称是：

LightCCI topology + CCI-derived N feature + GRN-derived G feature + block KL PIJ.

这个表达式不是一个需要计算的代数公式，而是方法组成说明。LightCCI topology 指同一时间点内的 CCI 图结构；N feature 指从该图中分解出的网络角色表示；G feature 指由表达和 GRN 共同生成的调控状态；block KL PIJ 指分别计算两类特征的 KL 成本后再融合并生成转移概率。

必须避免的误解：当前方法并没有用 GRN 直接重写 LightCCI 的 CCI 边，也没有把 GRN 边与 CCI 边直接相加。GRN 在当前最好方法中主要作为 unit-level feature 进入 PIJ 成本，而不是作为新的 CCI 邻接。

23.2 整套方法处理的对象：时间、层级、side 与 unit

一次 vertical 实验会给定一个 lower 层和一个 upper 层。例如，lower 层可以是 spot，upper 层可以是 `seurat_k40`。程序对同一个时间对分别在 lower side 和 upper side 上构造 PIJ，然后分别计算 EI。

为了避免后文符号混乱，本章首次定义全部基础索引：

- 用 t_s 表示源时间点，字母 s 来自 source；

- 用 t_t 表示目标时间点, 字母 t 来自 target;
- 用 ℓ 表示当前正在处理的层级, 可能是 lower 层, 也可能是 upper 层;
- 用 n_s 表示层级 ℓ 在源时间点 t_s 的 unit 数;
- 用 n_t 表示层级 ℓ 在目标时间点 t_t 的 unit 数;
- 用 $i \in \{1, \dots, n_s\}$ 表示一个源 unit 的行索引;
- 用 $j \in \{1, \dots, n_t\}$ 表示一个目标 unit 的行索引。

于是, 一个跨时间成本矩阵和一个 PIJ 矩阵都具有 $n_s \times n_t$ 的形状: 行代表源 unit, 列代表目标 unit。程序会对 lower side 做一遍完整过程, 再对 upper side 做一遍相同过程。

23.3 一条龙流程图

对任意一个 side、任意一个时间对, 完整流程可以写成:

$$\begin{aligned} (A_s^{\text{CCI}}, A_t^{\text{CCI}}) &\longrightarrow (N_s, N_t) \longrightarrow C^N, \\ (X_s, W_s; X_t, W_t) &\longrightarrow (G_s, G_t) \longrightarrow C^G, \\ (C^N, C^G) &\longrightarrow C^{\text{full}} \longrightarrow P \longrightarrow EI. \end{aligned}$$

这里首次出现的符号含义如下: A_s^{CCI} 和 A_t^{CCI} 分别是源、目标时间点的 CCI 邻接矩阵; N_s 和 N_t 是从两张 CCI 邻接中提取的 N 特征; C^N 是 N block 的跨时间 KL 成本; X_s 和 X_t 是源、目标时间点的 unit-by-gene 表达矩阵; W_s 和 W_t 是源、目标时间点的 GRN 邻接; G_s 和 G_t 是最终投影后的 GRN state; C^G 是 G block 的跨时间 KL 成本; C^{full} 是两个 block 融合后的总成本; P 是逐行归一化后的 PIJ; EI 是由该 PIJ 计算的有效信息。

为什么需要两条并行支路? 因为 CCI 邻接描述“unit 在通讯网络中处于什么结构角色”, 而 GRN state 描述“unit 当前处于什么调控状态”。前者是网络拓扑信息, 后者是表达条件化的调控信息。当前方法不在原始高维空间直接硬拼二者, 而是先让二者各自形成一张可比较的跨时间成本, 再在本层融合。

24. 第一部分: LightCCI 怎样提供 CCI 网络骨架

24.1 CCI 邻接的矩阵形式

对层级 ℓ 和时间点 t , LightCCI 构造:

$$A_{\ell,t}^{\text{CCI}} \in \mathbb{R}_{\geq 0}^{n_{\ell,t} \times n_{\ell,t}}.$$

这里, $A_{\ell,t}^{\text{CCI}}$ 是 CCI 邻接矩阵; $\mathbb{R}_{\geq 0}$ 表示所有非负实数; $n_{\ell,t}$ 表示层级 ℓ 在时间点 t 的 unit 数。矩阵元素

$$A_{\ell,t,ab}^{\text{CCI}}$$

表示同一时间点内从 unit a 指向 unit b 的通讯强度，其中 a 和 b 都是该层级 unit 的索引。

矩阵之所以是方阵，是因为发送者与接收者都来自同一个层级的同一组 unit。矩阵可以不对称，因为 $a \rightarrow b$ 与 $b \rightarrow a$ 的通讯方向和强度不必相同。

24.2 多张 ligand–receptor 矩阵如何合成总 CCI

若 COMMOT 提供了多张 ligand–receptor 通讯矩阵，则总邻接写为：

$$A_{\ell,t}^{\text{raw}} = \sum_{p=1}^{L_{\ell,t}} S_{\ell,t}^{(p)}.$$

这里， $A_{\ell,t}^{\text{raw}}$ 是尚未清洗的总通讯矩阵； $L_{\ell,t}$ 是该层级该时间点可用的 ligand–receptor 对数量； p 是 ligand–receptor 对的索引； $S_{\ell,t}^{(p)}$ 是第 p 个 ligand–receptor 对对应的 unit-by-unit 通讯矩阵。

为什么采用求和？因为不同 ligand–receptor 通道可以共同支持同一条 unit-to-unit 通讯。求和保留“总通讯质量”的含义，而不是只取最大通道或平均通道。若数据目录已经有总 CCI 文件，代码直接读取总矩阵，不重复求和。

24.3 非负清洗和阈值处理

对任一原始边权 z ，LightCCI 使用逐元素清洗：

$$\mathcal{T}_{\eta}(z) = \begin{cases} 0, & z < \eta \text{ 或 } z \text{ 不是有限数,} \\ z, & z \geq \eta \text{ 且 } z \text{ 为有限非负数.} \end{cases}$$

这里， $\mathcal{T}_{\eta}$ 是阈值清洗算子； $\eta \geq 0$ 是配置中的 CCI 最小阈值； z 是单个原始 CCI 边权；有限数表示不是 NaN、正无穷或负无穷。

最终邻接元素为：

$$A_{\ell,t,ab}^{\text{CCI}} = \mathcal{T}_{\eta}(A_{\ell,t,ab}^{\text{raw}}).$$

这里， $A_{\ell,t,ab}^{\text{raw}}$ 是清洗前从 unit a 到 unit b 的总通讯分数。这样写的目的，是让后续 NMF 接收稳定的非负输入，同时去掉数值异常和低于阈值的弱噪声边。

LightCCI 在这一阶段只构造同一时间点内的 CCI 图。它没有产生跨时间转移，也没有直接计算 EI。跨时间关系是在后面的 N/G 特征和 KL 成本中建立的。

25. 第二部分：CCI 邻接怎样变成 N 特征

25.1 为什么不能直接比较两张邻接矩阵的行

源时间点与目标时间点可能具有不同的 unit 数，尤其是 spot 层。即使 unit 数相同，邻接矩阵的列仍然表示各自时间点中的接收对象，不能简单把源矩阵第 i 行和目标矩阵第 j 行当成处于同一坐标系的向量。

NMF 的作用不是简单降维，而是为一个时间对共同学习一套潜在通讯角色，使源、目标 unit 都能在同一套角色坐标中表示。当前方法只使用时间对 t_s 和 t_t 的两张图，不读取其他时间点，因此是 pairwise NMF。

25.2 spot side: 共享核心的有向 NMF

在 spot side，源和目标 CCI 邻接分别记为：

$$A_s \in \mathbb{R}_{\geq 0}^{n_s \times n_s}, \quad A_t \in \mathbb{R}_{\geq 0}^{n_t \times n_t}.$$

这里， A_s 是源时间点 spot CCI 邻接； A_t 是目标时间点 spot CCI 邻接； n_s 与 n_t 分别是两个时间点的 spot 数。

给定 NMF 秩 r ，当前默认 $r = 5$ ，算法寻找：

$$U_s, V_s \in \mathbb{R}_{\geq 0}^{n_s \times r}, \quad U_t, V_t \in \mathbb{R}_{\geq 0}^{n_t \times r}, \quad B \in \mathbb{R}_{\geq 0}^{r \times r}.$$

这里， U_s 和 U_t 是源、目标 spot 的发送角色系数； V_s 和 V_t 是源、目标 spot 的接收角色系数； B 是两个时间点共享的角色交互核心； r 是潜在角色数量。

两张图被近似为：

$$A_s \approx U_s B V_s^T, \quad A_t \approx U_t B V_t^T.$$

这里，上标 T 表示矩阵转置。该近似的优化目标是：

$$\min_{U_s, V_s, U_t, V_t, B \geq 0} \|A_s - U_s B V_s^T\|_F^2 + \|A_t - U_t B V_t^T\|_F^2.$$

这里， $\|M\|_F$ 表示矩阵 M 的 Frobenius 范数，即矩阵所有元素平方和的平方根；条件 $U_s, V_s, U_t, V_t, B \geq 0$ 表示所有因子逐元素非负。

为什么共享 B ？因为 B 定义“发送角色如何连接接收角色”的共同语法。若源和目标分别学习完全独立的核心，源时间点的第一个角色与目标时间点的第一个角色可能毫无对应。共享核心把两个时间点约束在同一潜在结构系统中，而 U 和 V 仍允许各个 spot 的角色强度随时间变化。

spot 的 N 特征采用发送与接收角色拼接：

$$N_s = [U_s \mid V_s], \quad N_t = [U_t \mid V_t].$$

这里， N_s 和 N_t 分别是源、目标 spot 的 N 特征矩阵；符号 $[\cdot | \cdot]$ 表示按列拼接。因而：

$$N_s \in \mathbb{R}^{n_s \times 2r}, \quad N_t \in \mathbb{R}^{n_t \times 2r}.$$

默认 $r = 5$ 时，每个 spot 的 N 特征有 $2r = 10$ 维。之所以拼接而不相加，是因为发送角色和接收角色是两个不同方向的网络属性，相加会使二者互相抵消或失去可辨识度。

25.3 固定节点 domain side: 普通 pairwise joint NMF

对列空间能够直接对齐的非 spot 层，算法寻找：

$$W_s \in \mathbb{R}_{\geq 0}^{n_s \times r}, \quad W_t \in \mathbb{R}_{\geq 0}^{n_t \times r}, \quad H \in \mathbb{R}_{\geq 0}^{r \times m}.$$

这里， W_s 和 W_t 是源、目标 unit 的低维系数； H 是两个时间点共享的网络基底； m 是两张输入邻接共同的列数； r 仍是 NMF 秩。

近似关系为：

$$A_s \approx W_s H, \quad A_t \approx W_t H.$$

相应优化目标为：

$$\min_{W_s, W_t, H \geq 0} \|A_s - W_s H\|_F^2 + \|A_t - W_t H\|_F^2.$$

最终定义：

$$N_s = W_s, \quad N_t = W_t.$$

共享 H 的理由与 spot 中共享 B 相同：它让两个时间点的潜在分量具有共同语义。此时 N 特征维数为 r ，默认是 5 维。

25.4 N 特征的 pairwise z-score

NMF 的不同列可能具有完全不同的数值尺度。对 N 特征第 k 列，先将源和目标行拼在一起，计算：

$$\mu_k = \frac{\sum_{i=1}^{n_s} N_{s,ik} + \sum_{j=1}^{n_t} N_{t,jk}}{n_s + n_t}.$$

这里， μ_k 是第 k 个 N 分量在源、目标合并样本中的均值； $N_{s,ik}$ 是源 unit i 的第 k 个 N 分量； $N_{t,jk}$ 是目标 unit j 的第 k 个 N 分量； k 的范围是 1 到 N 特征维数 d_N 。

再计算合并标准差：

$$\sigma_k = \sqrt{\frac{\sum_{i=1}^{n_s} (N_{s,ik} - \mu_k)^2 + \sum_{j=1}^{n_t} (N_{t,jk} - \mu_k)^2}{n_s + n_t}}.$$

这里， σ_k 是第 k 列的标准差。标准化后的 N 特征为：

$$Z_{s,ik}^N = \frac{N_{s,ik} - \mu_k}{\sigma_k}, \quad Z_{t,jk}^N = \frac{N_{t,jk} - \mu_k}{\sigma_k}.$$

这里， Z_s^N 和 Z_t^N 是标准化后的源、目标 N 特征。若 $\sigma_k = 0$ ，代码把该列输出为零，避免除零。

为什么均值和标准差必须由源、目标共同拟合？因为若两个时间点分别标准化，每一列都会在各自时间点被强制变成零均值和单位方差，可能人为抹去时间变化。联合拟合让源和目标处在同一个数值坐标系中。

26. 第三部分：表达与 GRN 怎样变成 G 特征

26.1 GRN 原始边与基因集合

对一个时间点，GRN 中从基因 g 指向基因 h 的原始权重记为：

$$\omega_{g \rightarrow h} \in \mathbb{R}.$$

这里， g 是 regulator 基因索引； h 是 target 基因索引； $\omega_{g \rightarrow h}$ 是有向调控边的原始权重。

当前代码首先取绝对值：

$$\bar{\omega}_{g \rightarrow h} = |\omega_{g \rightarrow h}|.$$

这里， $\bar{\omega}_{g \rightarrow h}$ 是用于后续计算的非负调控强度。取绝对值的原因是当前 GRN state 主要表达“调控连接强度”，并保证后续传播为非负；代价是激活与抑制符号被丢失。这是当前模型边界，不代表生物学上正负调控没有区别。

只保留表达矩阵中实际出现的 regulator 和 target，并对同一个 regulator 的候选 target 按权重排序，保留前 K 个 target。当前默认：

$$K = 50.$$

这里， K 是每个 regulator 最多保留的 target 数。top- K 的目的，是避免大量弱边把 GRN state 变成近似全局平均，同时控制计算量。

26.2 GRN 邻接的行归一化

设最终保留的基因数为 m ，构造：

$$W \in \mathbb{R}_{\geq 0}^{m \times m}.$$

这里， W 是 GRN 邻接矩阵； m 是对齐后 regulator 与 target 并集中的基因数；元素 W_{gh} 表示从基因 g 到基因 h 的非负调控权重。

对每个 regulator 行做归一化：

$$W_{gh} = \frac{\bar{\omega}_{g \rightarrow h}}{\sum_{q=1}^m \bar{\omega}_{g \rightarrow q}} \quad \text{当分母大于零时.}$$

这里， q 是 target 基因求和索引；分母是 regulator g 的全部保留出边权重之和。若某一行没有正权重，该行保持为零。

为什么行归一化？若不归一化，一个总出边权特别大的 regulator 会仅凭总权重尺度支配 GRN state。行归一化后，每一行更接近“该 regulator 的调控质量在各 target 间如何分配”，强调相对目标结构，而不是原始总量。

26.3 unit-by-gene 表达矩阵

对同一层级同一时间点，表达矩阵记为：

$$X \in \mathbb{R}_{\geq 0}^{n \times m}.$$

这里， X 是 unit-by-gene 表达矩阵； n 是该时间点当前层级的 unit 数； m 是与 GRN 对齐后的基因数；元素 X_{ug} 表示 unit u 中基因 g 的非负表达值。

若某个 unit 在表达文件中缺失，重索引后该行补零；NaN 和无穷值被转成零；负值被截到零。这样做是为了让表达门控具有明确的“强度”含义。

对单个 unit u ，把其表达行写成列向量：

$$x_u \in \mathbb{R}_{\geq 0}^m.$$

这里， x_u 是 unit u 的基因表达向量，向量第 g 个元素是 X_{ug} 。

26.4 双端表达门控：regulator side

先计算：

$$p_u^{\text{reg}} = W x_u.$$

这里， $p_u^{\text{reg}} \in \mathbb{R}_{\geq 0}^m$ 是 regulator program；其第 g 个元素为：

$$p_{u,g}^{\text{reg}} = \sum_{h=1}^m W_{gh} x_{u,h}.$$

这里， $p_{u,g}^{\text{reg}}$ 表示 regulator g 的已知 target 在 unit u 中的加权表达汇总； h 是 target 基因求和索引。

注意， $W x_u$ 并不是“把 regulator 表达向 target 传播”的传统写法。由于 W 的行是 regulator、列是 target， $W x_u$ 的第 g 个分量是在查看 regulator g 的 target program 是否在当前 unit 中被表达。

随后进行逐元素表达门控：

$$s_u^{\text{reg}} = x_u \odot p_u^{\text{reg}}.$$

这里， $s_u^{\text{reg}} \in \mathbb{R}_{\geq 0}^m$ 是 regulator-side state；符号 $\odot$ 表示逐元素乘法。第 g 个分量为：

$$s_{u,g}^{\text{reg}} = x_{u,g} \left(\sum_{h=1}^m W_{gh} x_{u,h} \right).$$

这个公式为什么要乘 $x_{u,g}$ ？因为仅有 target program 高并不意味着 regulator g 自身在当前 unit 中活

跃。乘法要求两个条件同时成立：regulator 自身表达较高，并且它的 target program 也较高。只要其中一端接近零，该 regulator-side state 就会受到抑制。

26.5 双端表达门控：target side

另一条支路先计算：

$$p_u^{\text{tar}} = W^{\text{T}} x_u.$$

这里， $p_u^{\text{tar}} \in \mathbb{R}_{\geq 0}^m$ 是 target program； W^{T} 是 GRN 邻接的转置。其第 h 个元素为：

$$p_{u,h}^{\text{tar}} = \sum_{g=1}^m W_{gh} x_{u,g}.$$

这里， $p_{u,h}^{\text{tar}}$ 汇总所有 regulator 对 target h 的权重，并用这些 regulator 在 unit u 中的表达加权； g 是 regulator 基因求和索引。

再做逐元素门控：

$$s_u^{\text{tar}} = x_u \odot p_u^{\text{tar}}.$$

这里， $s_u^{\text{tar}} \in \mathbb{R}_{\geq 0}^m$ 是 target-side state。第 h 个分量为：

$$s_{u,h}^{\text{tar}} = x_{u,h} \left(\sum_{g=1}^m W_{gh} x_{u,g} \right).$$

这要求 target h 自身被表达，同时其上游 regulator program 也被表达。于是 regulator side 更关注“一个 regulator 与其下游 program 是否共同活跃”，target side 更关注“一个 target 与其上游调控输入是否共同活跃”。

双端门控的核心不是简单把表达和 GRN 相加，而是用乘法构造一种“AND”条件：表达状态与网络关系必须同时支持该分量，state 才会大。这样得到的是 expression-conditioned GRN state，而不是静态 GRN 拓扑本身。

26.6 为什么需要确定性随机投影

原始 s_u^{reg} 与 s_u^{tar} 都有 m 维，而 m 可能达到数千。直接对所有源 unit 与目标 unit 计算高维 KL 成本会增加内存和计算量，而且不同时间点保留的基因集合可能不完全相同。

当前代码为每个基因和每个角色生成一个确定性的随机向量。设投影维数为 d_G ，当前默认：

$$d_G = 64.$$

这里， d_G 是最终 G feature 的维数。

regulator 与 target 使用两张投影矩阵：

$$R^{\text{reg}}, R^{\text{tar}} \in \mathbb{R}^{m \times d_G}.$$

这里， R^{reg} 是 regulator-side 投影矩阵； R^{tar} 是 target-side 投影矩阵。每一行由“全局 seed、角色字符串、基因名”的 SHA256 哈希确定，再生成高斯随机向量。当前全局 projection seed 为 20260713。

投影元素的尺度约为：

$$\text{Std}(R_{gk}^{\text{role}}) = \frac{1}{\sqrt{d_G}}.$$

这里， R_{gk}^{role} 表示某一角色投影矩阵中基因 g 到投影维度 k 的元素；Std 表示标准差；role 可以是 regulator 或 target。使用 $1/\sqrt{d_G}$ 是为了让投影后向量的典型尺度不随输出维数无限增大。

最终 unit u 的 G feature 为：

$$g_u = (s_u^{\text{reg}})^{\top} R^{\text{reg}} + (s_u^{\text{tar}})^{\top} R^{\text{tar}}.$$

这里， $g_u \in \mathbb{R}^{d_G}$ 是 unit u 的最终投影 GRN state； $(s_u^{\text{reg}})^{\top}$ 和 $(s_u^{\text{tar}})^{\top}$ 是两个 state 的行向量形式。

为什么使用确定性随机投影，而不是训练一个编码器？

- 它不需要跨时间训练，因此不会引入额外的监督目标或时间信息泄露；
- 相同基因、相同角色、相同 seed 永远获得相同投影向量，使不同时间点的坐标可复现；
- 即使不同时间点基因集合略有差异，最终输出维数仍然固定为 d_G ；
- 它把计算复杂度从数千维降到 64 维。

为什么 regulator 与 target 投影后采用相加？当前代码希望把两种角色压到同一个固定维度，避免输出变成 $2d_G$ 。由于两张投影矩阵由不同 role 哈希生成，它们并不是使用同一组方向简单相加。不过，相加也意味着两类信息不再能够被后续完全分开识别，这是当前实现的一个解释边界。

26.7 G 特征的 pairwise z-score

对源时间点所有 unit 得到：

$$G_s \in \mathbb{R}^{n_s \times d_G},$$

对目标时间点所有 unit 得到：

$$G_t \in \mathbb{R}^{n_t \times d_G}.$$

这里， G_s 与 G_t 分别是源、目标的原始投影 GRN state 矩阵。

代码把 G_s 和 G_t 纵向拼接，对每一列共同计算均值与标准差，再得到标准化矩阵：

$$Z_s^G \in \mathbb{R}^{n_s \times d_G}, \quad Z_t^G \in \mathbb{R}^{n_t \times d_G}.$$

这里， Z_s^G 和 Z_t^G 是 pairwise z-score 后的 G 特征。具体公式与 N block 的 z-score 完全相同，只是特征维数从 d_N 换成 d_G 。

为什么 G 也要标准化？随机投影的不同维度可能具有不同方差，且 regulator/target state 总强度会随时间变化。联合 z-score 让每个投影方向在当前时间对内处于相近尺度，使 KL 更关注各维度的相对模式，而不是某一维的绝对方差。

27. 第四部分：N 与 G 怎样分别变成 KL 成本

27.1 为什么 KL 之前先把 feature 变成概率分布

KL 散度比较的是两个概率分布，而标准化后的 N 或 G feature 可以有正值、零值和负值，不能直接作为概率。因此，代码对每一个 unit 的 feature 行做 softmax。

以 N block 为例，对源 unit i 的第 k 个特征分量定义：

$$\pi_{s,ik}^N = \frac{\exp(Z_{s,ik}^N / \beta)}{\sum_{h=1}^{d_N} \exp(Z_{s,ih}^N / \beta)}.$$

这里， $\pi_{s,ik}^N$ 是源 unit i 在 N feature 第 k 维上的概率质量； $Z_{s,ik}^N$ 是标准化 N 特征； $\beta > 0$ 是 feature softmax 温度； h 是 softmax 分母中的特征维度求和索引； d_N 是 N feature 维数。当前代码使用：

$$\beta = 0.05.$$

目标 unit j 的 N 概率表示为：

$$\pi_{t,jk}^N = \frac{\exp(Z_{t,jk}^N / \beta)}{\sum_{h=1}^{d_N} \exp(Z_{t,jh}^N / \beta)}.$$

这里， $\pi_{t,jk}^N$ 是目标 unit j 在 N feature 第 k 维上的概率质量。

为什么使用 softmax？它完成三件事：第一，把任意实数特征变成非负；第二，让一行总和为 1；第三，把向量解释成“这个 unit 的潜在角色质量如何分配”。较小的 β 会放大最大特征分量，使分布更尖锐；较大的 β 会让分布更均匀。

代码在指数前先减去每一行最大值，避免数值溢出；softmax 后再将概率裁剪到 $[\varepsilon, 1]$ ，其中：

$$\varepsilon = 10^{-12}.$$

这里， ε 是避免出现 $\log 0$ 的极小正数。

27.2 N block 的有向 KL 成本

源 unit i 到目标 unit j 的 N block 成本为：

$$C_{ij}^N = D_{\text{KL}}(\pi_{s,i}^N \parallel \pi_{t,j}^N) = \sum_{k=1}^{d_N} \pi_{s,ik}^N \log \frac{\pi_{s,ik}^N}{\pi_{t,jk}^N}.$$

这里， C_{ij}^N 是源 unit i 到目标 unit j 的 N 成本； $D_{\text{KL}}(p \parallel q)$ 表示从分布 p 到分布 q 的 Kullback–Leibler 散度； $\pi_{s,i}^N$ 表示源 unit i 的整行 N 概率向量； $\pi_{t,j}^N$ 表示目标 unit j 的整行 N 概率向量； $\log$ 是自然对数。

为什么使用 $D_{\text{KL}}(\text{source} \parallel \text{target})$ 而不是对称距离？跨时间 PIJ 本身是有向的：我们问的是源状态能否被目标状态解释。KL 的方向性保留了 source-to-target 的不对称关系。交换 i 和 j 通常会得到不同数值，这与时间方向一致。

把所有 i, j 组合计算后得到：

$$C^N \in \mathbb{R}_{\geq 0}^{n_s \times n_t}.$$

这里， C^N 是完整 N block 成本矩阵。数值越小，表示源 unit 与目标 unit 的网络角色分布越相似。

27.3 G block 的 softmax 与 KL 成本

对标准化 G 特征采用同一 softmax 温度 β ：

$$\pi_{s,ik}^G = \frac{\exp(Z_{s,ik}^G / \beta)}{\sum_{h=1}^{d_G} \exp(Z_{s,ih}^G / \beta)}, \quad \pi_{t,jk}^G = \frac{\exp(Z_{t,jk}^G / \beta)}{\sum_{h=1}^{d_G} \exp(Z_{t,jh}^G / \beta)}.$$

这里， $\pi_{s,ik}^G$ 和 $\pi_{t,jk}^G$ 分别是源 unit i 与目标 unit j 在 G feature 第 k 维上的概率质量；此处 $k, h \in \{1, \dots, d_G\}$ 。

G block 成本为：

$$C_{ij}^G = \sum_{k=1}^{d_G} \pi_{s,ik}^G \log \frac{\pi_{s,ik}^G}{\pi_{t,jk}^G}.$$

这里， C_{ij}^G 是源 unit i 到目标 unit j 的 GRN state KL 成本。所有 unit 对构成：

$$C^G \in \mathbb{R}_{\geq 0}^{n_s \times n_t}.$$

为什么 N 与 G 分别做 softmax 和 KL，而不是先把两种 feature 拼接？因为 N 通常只有 5 或 10 维，而 G 默认有 64 维，二者来源和数值结构完全不同。若直接拼接后统一 softmax，维数更多的 G

block 可能天然分走更多概率质量，且无法单独控制两种信息源。分块计算让 N 与 G 各自先形成一张同形状成本，再在成本层公平融合。

28. 第五部分：两个 KL block 怎样融合成总成本

28.1 为什么原始 C^N 和 C^G 不能直接相加

即使两个成本矩阵形状相同，它们的绝对数值范围也可能相差很大。N block 的维数、分布尖锐程度和 G block 不同。若直接写成 $0.5C^N + 0.5C^G$ ，数值范围更大的 block 会在实际中占主导，名义上的 0.5/0.5 并不代表相同影响力。

因此，当前完整方法先分别进行 5%–95% 稳健归一化。

28.2 单个成本矩阵的稳健归一化

对任意成本矩阵 C ，定义其有限元素第 5 百分位数和第 95 百分位数：

$$q_5(C), \quad q_{95}(C).$$

这里， $q_5(C)$ 是成本矩阵有限元素的第 5 百分位数； $q_{95}(C)$ 是第 95 百分位数。

归一化后的元素为：

$$\tilde{C}_{ij} = \text{clip}\left(\frac{C_{ij} - q_5(C)}{q_{95}(C) - q_5(C)}, 0, 1\right).$$

这里， $\tilde{C}_{ij}$ 是稳健归一化后的成本； $\text{clip}(y, 0, 1)$ 表示将小于 0 的 y 设为 0，将大于 1 的 y 设为 1，其余保持不变。

为什么使用 5% 和 95%，而不是 min–max？极少数异常大的 KL 成本可能拉伸整个尺度，使绝大多数正常成本挤在接近零的小范围。分位数归一化把典型成本区间映射到 $[0, 1]$ ，并对两端离群值裁剪。若两个分位数退化为相同值，代码退回 min–max；若 min–max 仍退化，输出零矩阵。

分别得到：

$$\tilde{C}^N, \quad \tilde{C}^G.$$

这里， $\tilde{C}^N$ 是 N block 的归一化成本； $\tilde{C}^G$ 是 G block 的归一化成本。

28.3 当前完整方法的加权融合

定义 N 与 G 的权重：

$$w_N \geq 0, \quad w_G \geq 0, \quad w_N + w_G = 1.$$

这里， w_N 是 N block 权重； w_G 是 G block 权重。当前默认：

$$w_N = 0.5, \quad w_G = 0.5.$$

总成本为：

$$C_{ij}^{\text{full}} = w_N \tilde{C}_{ij}^N + w_G \tilde{C}_{ij}^G.$$

这里， C_{ij}^{full} 是源 unit i 到目标 unit j 的最终融合成本； C^{full} 是形状为 $n_s \times n_t$ 的总成本矩阵。

为什么是加权平均？它把两类证据解释为两个并行评价标准：CCI 结构角色越相似， $\tilde{C}_{ij}^N$ 越小；GRN 调控状态越相似， $\tilde{C}_{ij}^G$ 越小。只有两者共同较小时，总成本才会非常小。若其中一项大，候选对应会受到惩罚。

这不是严格逻辑上的 AND，因为一项较小仍可部分补偿另一项；它更像一个软共识。0.5/0.5 是对称起点，不等于已经证明 N 与 G 的生物重要性相同。

29. 第六部分：总成本怎样变成 PIJ

29.1 指数核：低成本变成高相似度

对每一个源 unit i 与目标 unit j ，先计算：

$$K_{ij} = \exp\left(-\frac{C_{ij}^{\text{full}}}{\tau}\right).$$

这里， K_{ij} 是尚未归一化的相似性核； $\tau > 0$ 是 PIJ 温度；当前默认 $\tau = 1.0$ ； $\exp$ 是自然指数函数。

为什么使用负指数？成本越小， K_{ij} 越接近 1；成本越大， K_{ij} 越接近 0。指数核保证所有候选对应获得非负权重，并以平滑方式把“距离”转换成“相似度”。

τ 决定成本差异被放大的程度。较小的 τ 使最低成本候选更突出，PIJ 更尖锐；较大的 τ 使不同候选权重更接近，PIJ 更平滑。

29.2 逐行归一化：得到条件转移概率

最终 PIJ 为：

$$P_{ij} = \frac{K_{ij}}{\sum_{q=1}^{n_t} K_{iq}}.$$

这里， P_{ij} 是给定源 unit i 后转向目标 unit j 的条件概率； q 是目标 unit 求和索引； $P \in [0, 1]^{n_s \times n_t}$ 是 PIJ 矩阵。

每一行满足：

$$\sum_{j=1}^{n_t} P_{ij} = 1.$$

这里，求和等式表示每个源 unit 的所有目标候选构成一个完整概率分布。

为什么逐行归一化，而不是把整张矩阵归一化？因为当前 PIJ 的语义是“给定源状态，目标状态的条件分布”。每个源 unit 都应独立拥有总质量 1，不能让 unit 数量或其他行的总权重影响本行概率。

29.3 用一个源 unit 口语化理解整条链

对某一个源 unit i ，程序会对每个目标 unit j 做两次比较：

1. 比较 i 与 j 的 CCI 网络角色分布，得到 C_{ij}^N ；
2. 比较 i 与 j 的表达条件化 GRN 状态分布，得到 C_{ij}^G ；
3. 分别把两张成本按各自典型范围缩放；
4. 取两项加权平均得到 C_{ij}^{full} ；
5. 将低成本候选转成高指数权重；
6. 在该源 unit 的所有目标候选间归一化，得到整行 $P_{i,:}$ 。

因此，PIJ 不是某一条 CCI 边，也不是某一条 GRN 边。它是一个跨时间条件概率，综合了当前源 unit 与所有目标 unit 在两种 feature 空间中的相对匹配程度。

30. 第七部分：PIJ 怎样变成 EI 与 EI gain

30.1 目标边际分布

假设对每一个源 unit 施加均匀干预，即每一行被同等选择。目标状态的平均分布定义为：

$$\bar{P}_j = \frac{1}{n_s} \sum_{i=1}^{n_s} P_{ij}.$$

这里， $\bar{P}_j$ 是目标 unit j 在均匀源干预下的边际概率； n_s 是源 unit 数； P_{ij} 是 PIJ 元素。

30.2 目标边际熵

目标边际熵为：

$$H(J) = - \sum_{j=1}^{n_t} \bar{P}_j \log_2(\bar{P}_j + \epsilon_{EI}).$$

这里， $H(J)$ 是目标状态随机变量 J 的熵； $\log_2$ 是以 2 为底的对数； $\epsilon_{EI} = 10^{-12}$ 是避免对零取对数的数值稳定项。

该量越大，表示总体上目标质量分散在更多目标状态中。

30.3 给定源状态后的平均条件熵

条件熵为：

$$H(J | I) = - \frac{1}{n_s} \sum_{i=1}^{n_s} \sum_{j=1}^{n_t} P_{ij} \log_2(P_{ij} + \epsilon_{EI}).$$

这里， $H(J | I)$ 是知道源状态随机变量 I 后，目标状态 J 的平均不确定性。若每一行 PIJ 都很尖锐，条件熵较低；若每一行都接近均匀，条件熵较高。

30.4 有效信息

当前代码的 EI 定义为：

$$EI = H(J) - H(J | I).$$

这里， EI 的单位是 bit，因为使用了 $\log_2$ 。它等价于在均匀源干预下源状态与目标状态之间的互信息。

为什么这个差值能衡量有效信息？ $H(J)$ 衡量目标整体可用状态的多样性； $H(J | I)$ 衡量给定具体源状态后仍然剩余多少不确定性。若不同源 unit 的 PIJ 行几乎完全相同，知道源状态并不能减少目标不确定性， EI 接近零。若不同源状态对应不同而稳定的目标分布，知道源状态显著降低不确定性， EI 增大。

30.5 lower EI、upper EI 与 EI gain

对同一个时间对，程序在 lower side 得到：

$$P^{\text{lower}}, \quad EI_{\text{lower}}.$$

这里， P^{lower} 是 lower 层 PIJ； EI_{lower} 是由该 PIJ 计算的微观层有效信息。

在 upper side 得到：

$$P^{\text{upper}}, \quad EI_{\text{upper}}.$$

这里， P^{upper} 是 upper 层 PIJ； EI_{upper} 是由该 PIJ 计算的宏观层有效信息。

最终：

$$EI_{\text{gain}} = EI_{\text{upper}} - EI_{\text{lower}}.$$

这里， EI_{gain} 是宏观层相对于微观层的有效信息增量。若 $EI_{\text{gain}} > 0$ ，说明 upper 层在当前 PIJ 构造下具有更高的源到目标可辨识信息；若小于零，则该粗粒化没有带来 EI 优势。

EI gain 的正负必须与 EI 的绝对量级一起看。若 lower EI 和 upper EI 都极接近零，正 gain 可能只是两个极小数之差。因此后续分析还应检查 PIJ 行熵、最大概率和 EI 绝对值。

31. 当前方法每一步为什么这样设计

设计	目的、收益与边界
LightCCI 只保留 CCI 邻接	避免直接构造巨型跨细胞 GRN-CCI 稠密网络；把网络骨架与 PIJ feature 分开。边界是 GRN 没有直接改变 CCI 边。
CCI 使用非负边权	适配非负分解并把边解释成通讯强度。负值或符号信息不在该网络中表达。
pairwise NMF	只使用当前时间对，令源和目标共享潜在角色语义，同时避免把其他时间点引入该对。
spot 分开发送与接收角色	CCI 是有向网络；拼接 U 与 V 保留 outgoing 与 incoming 两类结构。
GRN 每 regulator top-50	控制噪声和计算量，避免大量弱 target 令 state 过度平均。
GRN 行归一化	让每个 regulator 更强调 target 分配结构，降低总出边权尺度差异。
双端表达门控	要求基因本身表达与其上游或下游 program 同时活跃，使 G 成为状态化 GRN，而非静态拓扑。

设计	目的、收益与边界
确定性随机投影到 64 维	控制维度、保证可复现、避免训练和时间泄露；边界是投影会压缩细节。
N 与 G 分别 z-score	令每个时间对的源、目标处于共同尺度，避免某些列仅因方差大而主导。
feature 行 softmax	把实数 feature 转成概率分布，让 KL 比较相对组成；会弱化绝对幅值。
source-to-target KL	保留时间方向，衡量源分布被目标分布解释的代价；KL 不对称。
N/G 分别稳健归一化	让两个 block 在典型成本范围上具有可比较尺度；边界是绝对 KL 尺度被重标定。
0.5/0.5 加权平均	提供一个无偏向的初始融合点；并不证明两类信息生物重要性相等。
指数核与行归一化	将低成本变成高概率，并获得每个源状态独立的条件目标分布。
EI upper-minus-lower	直接衡量当前 PIJ 下宏观层是否获得信息优势；不能单独说明结果来自哪一种 feature。

32. 当前方法究竟是什么，又不是什么

32.1 它是什么

当前方法是一个**网络骨架固定、feature 成本融合**的跨时间 **PIJ** 方法：

- LightCCI CCI 图定义 unit 在同一时间点内的通讯结构；
- N feature 概括 unit 的通讯网络角色；
- G feature 概括表达条件化的调控状态；
- N/G block-KL 评价一个源 unit 与一个目标 unit 在两种信息上的匹配；
- PIJ 把全部候选匹配转成条件转移概率；
- EI 比较 lower 与 upper PIJ 的可辨识信息。

32.2 它不是什么

- 它不是把 CCI 邻接与 GRN 邻接直接相加的网络；
- 它不是把 N 和 G 直接拼成一个 feature 后计算一次 KL；
- 它不是对 G 再做一次 NMF；
- 它不是最优传输，因为当前 KL 路径使用指数核和行归一化，没有 Sinkhorn 边际约束；
- 它不是直接在 CCI 边或 GRN 边上计算 EI，EI 的直接输入是跨时间 PIJ；
- 它不能仅凭 Full 结果证明真实 GRN 必然有效，仍需要 N-only、G-only 和 shuffled-G 对照。

33. 默认参数与实际维度速查

参数或对象	当前默认	含义
NMF components r	5	spot 的 N 维数为 $2r = 10$ ；其他固定节点层通常为 $r = 5$ 。
GRN top- K	50	每个 regulator 最多保留 50 个 target。
G feature 维数 d_G	64	双端 GRN state 投影后的最终维数。
GRN projection seed	20260713	与基因名和 role 一起决定可复现随机投影。
GRN gate mode	double_end	同时构造 regulator-side 和 target-side 表达门控 state。
N block 权重 w_N	0.5	完整方法中 N 归一化成本的权重。
G block 权重 w_G	0.5	完整方法中 G 归一化成本的权重。
feature softmax 温度 β	0.05	控制 N/G feature 概率分布的尖锐程度。
PIJ 温度 τ	1.0	控制融合成本到转移概率的平滑程度。
成本归一化	5%–95%	N 与 G 两个成本矩阵分别独立做稳健缩放。
EI 数值稳定项 ϵ_{EI}	10^{-12}	防止熵公式中出现 $\log_2 0$ 。

34. 与阶段一四组实验的对应关系

完整原理明确后，阶段一四组实验可以理解为只对上述链条中的某一个部件做控制：

- **Full**：保留 $\tilde{C}^N$ 与 $\tilde{C}^G$ ，使用当前 0.5/0.5 融合；

- **N-only**: 保留相同 NMF、N softmax、N KL 和稳健归一化，只删除 G block；
- **G-only**: 保留相同 GRN state、G softmax、G KL 和稳健归一化，只删除 N block；
- **N+shuffled-G**: 保留真实 G 行向量集合和全部数学步骤，只打乱 G 行与 unit 身份的对应关系。

因此，阶段一不是重新发明方法，而是在当前完整链条中逐项关闭或破坏特定信息，以回答 Full 的好结果究竟来自真实 GRN 信息、N/G 互补，还是仅来自增加一个成本 block 与归一化。

35. 从代码入口追踪整条数学链

代码位置	对应数学职责
<code>networks/light_cci.py</code>	读取并清洗 CCI 邻接，生成每个时间点、每个层级的 LightCCI graph。
<code>networks/light_cci_grn.py</code>	在 LightCCI graph 不变的基础上读取表达和 GRN，将投影 G state 写入 graph metadata。
重构后同文件中的 GRN 私有函数	基因对齐、绝对权重、top-K、GRN 行归一化、双端表达门控和确定性随机投影。
<code>pij/compare/features.py</code>	对每个时间对构造 NMF-based N feature，并读取、对齐、标准化 G feature。
重构后的 <code>pij/compare/compare_N_kl.py</code>	计算 N KL、G KL、独立稳健归一化和加权 block cost。
<code>pij/compare/common.py</code>	调用方法私有 KL cost，执行指数核、PIJ 行归一化、导出与 diagnostics。
<code>metrics.py</code>	分别对 lower 与 upper PIJ 计算 EI，并输出 $EI_{\text{gain}} = EI_{\text{upper}} - EI_{\text{lower}}$

整套方法的最终口语化理解：每个时间点先有一张 CCI 图。NMF 告诉我们每个 unit 在这张通讯图中像哪一种发送者或接收者；表达门控 GRN 告诉我们每个 unit 当前有哪些调控 program 真正处于活跃状态。对一个源 unit 和一个目标 unit，方法分别问“网络角色像不像”和“调控状态像不像”，把两个不相似度转换到同一尺度后平均，再把所有目标候选归一化为转移概率。最后比较微观层与宏观层的这些转移概率，判断宏观层是否保留了更多可辨识的源到目标信息。